

Caracterización del desempeño y el coste computacional en algoritmos de similitud en bancos de imágenes de grafiti

Alberto Luengo Román¹

Departamento de Informática y Automática, Universidad de Salamanca, España
luengor@usal.es

Resumen Manual tracking of unauthorized graffiti becomes infeasible as image banks grow. This work designs, implements and experimentally evaluates a modular system to cluster an unlabelled dataset of graffiti photographs by visual similarity, aiming to characterize both clustering quality and the computational cost of its components as a function of bank size. The architecture chains swappable stages (segmentation, *embedding* extraction, storage, dimensionality reduction and clustering), and a benchmarking framework sweeps combinations of them, measuring per-stage time, memory and quality metrics for each run. The resulting base configuration (a *fine-tuned* DINOv2 style head + UMAP-10 + HDBSCAN + full image) is validated both on the dataset of 6416 graffiti images and on disjoint manually labelled subsets. The style head separates four styles with an adjusted Rand index of 0.722 (K-Means, oracle $k=4$), whereas author clustering stays near chance, exposing the insufficient per-author annotations in the test set. The scaling study shows that SQLite similarity search and ingestion are the dominant bottlenecks at this scale, while other factors such as the complexity of the reduction or clustering algorithms would only start to dominate at much larger scales. We conclude that intrinsic clustering-evaluation metrics can be misleading and that the choice of feature extractor must be anchored in supervised evidence.

Key words: graffiti, image clustering, deep *embeddings*, dimensionality reduction, HDBSCAN, similarity search, computational cost.

1 Introducción

1.1 Contexto y motivación

El grafiti no autorizado es un fenómeno urbano cotidiano que las administraciones municipales gestionan con un coste económico y social sostenido [1]. La diversidad estilística de los grafitis, desde firmas rápidas hasta piezas elaboradas, y la continua aparición de nuevos autores convierten su seguimiento manual en una tarea costosa y complicada. Taylor *et al.* [2] documentan que la falta de criterios homogéneos de clasificación entre quien reporta un grafiti y quien lo retira es un problema recurrente, y proponen una tipología explícita que ha demostrado ser útil para priorizar la retirada de grafitis ofensivos o no autorizados

según el tipo y la ofensividad del contenido. Esa tipología, sin embargo, sigue aplicándose de forma manual, y su extensión a bancos de imágenes crecientes no es viable sin apoyo automatizado.



Figura 1. Ejemplos de recortes de grafiti del conjunto de datos, que ilustran la diversidad estilística de los grafitis: desde firmas rápidas hasta piezas elaboradas.

En esta línea, trabajos recientes han abordado la detección y conteo automático de grafiti sobre imágenes captadas en condiciones no controladas [1,3], centrados en localizar grafitis y estimar su presencia geográfica, pero sin abordar su agrupación por similitud visual. Disponer de una agrupación automática por similitud visual habilita una gran cantidad de procedimientos de análisis que van desde el estilístico hasta el documental, incluyendo la priorización de la retirada o la trazabilidad de autores recurrentes sobre conjuntos de imágenes en los que la inspección manual es inviable.

1.2 Planteamiento del problema

El problema técnico abordado en este trabajo es el siguiente: dado un conjunto mayoritariamente no etiquetado de fotografías de grafiti captadas en condiciones reales, agruparlas automáticamente de modo que los grafitis visualmente similares queden en el mismo grupo. Este enunciado, aparentemente directo, encierra tres dificultades que condicionan tanto el diseño como la evaluación de la solución.

En primer lugar, no existe verdad fundamental canónica para la similitud entre grafitis: no hay un conjunto de etiquetas establecido sobre el que comparar los resultados. Además, el significado de «visualmente similar» no es del todo claro; no es evidente *a priori* si la similitud debe centrarse en el estilo, la autoría, el contenido o una combinación de estos factores. Para habilitar una evaluación

supervisada se ha etiquetado *ad hoc* un subconjunto reducido, que se utiliza como método de evaluación complementaria y no como referencia absoluta. En segundo lugar, el espacio de combinaciones de modelos de representación, técnicas de reducción de dimensionalidad y algoritmos de agrupamiento es amplio, y la literatura no ofrece una guía clara sobre qué componentes funcionan mejor en este dominio. En tercer lugar, el coste computacional de cada combinación escala con el tamaño del conjunto de imágenes, sin que exista una caracterización experimental previa específica para grafiti que permita anticipar dónde se sitúan los cuellos de botella.

1.3 Objetivos

Objetivo general Diseñar, implementar y evaluar experimentalmente un sistema modular de agrupamiento de imágenes de grafiti que permita caracterizar el coste computacional y la calidad del agrupamiento de combinaciones de segmentación, generación de representaciones vectoriales (*embeddings*), reducción de dimensionalidad y *clustering* en función del tamaño del conjunto.

Objetivos específicos

- Construir una arquitectura modular con interfaces intercambiables para cada etapa del flujo: segmentación, representación, almacenamiento, reducción y agrupamiento, que permita ensayar y comparar combinaciones de estas operaciones.
- Integrar modelos de representación que reflejen el estado de la cuestión: redes convolucionales preentrenadas, modelos fundacionales autosupervisados y supervisados por lenguaje natural, y detectores específicos de grafiti.
- Implementar un sistema capaz de recorrer un espacio de configuraciones definido por combinaciones de componentes y medir, para cada ejecución, tiempos por etapa y métricas de calidad de los clústeres resultantes.
- Caracterizar experimentalmente el coste computacional por etapa en función del tamaño del conjunto de imágenes (N), identificando cuellos de botella y escalado por componente.
- Evaluar la calidad del agrupamiento mediante métricas intrínsecas sobre el conjunto completo y métricas extrínsecas sobre el subconjunto etiquetado manualmente.
- Producir un análisis comparativo que identifique qué combinaciones ofrecen el mejor compromiso entre coste computacional y calidad de agrupamiento sobre el conjunto de datos disponible.

1.4 Alcance y limitaciones

El trabajo diseña e implementa el sistema modular y se centra en la evaluación comparativa de combinaciones de componentes del estado de la cuestión ya existentes, en el cálculo de métricas intrínsecas de calidad de *clustering*, en

la caracterización de tiempos por etapa y su escalado con N , y en una evaluación extrínseca limitada sobre el subconjunto etiquetado manualmente para este trabajo.

Conviene tener en cuenta dos limitaciones metodológicas. Primero, las etiquetas utilizadas para la evaluación supervisada han sido generadas a mano para este trabajo, sobre un subconjunto reducido y por un solo anotador, sin ningún tipo de validación externa o consenso entre anotadores. Así, las métricas extrínsecas obtenidas son indicativas y no concluyentes. Segundo, la evaluación extrínseca solo es directamente aplicable a las configuraciones que no hacen uso de segmentación automática. Esto se debe a que la segmentación automática extrae y genera nuevos recortes de las imágenes originales, que no corresponderán exactamente con los recortes etiquetados manualmente, lo que impide una comparación directa.

Por último, cabe destacar que este trabajo se centra exclusivamente en aspectos técnicos. El grafiti se aborda como un fenómeno visual neutro, sin entrar en consideraciones sociales, culturales o legales. El objetivo es desarrollar un sistema de agrupamiento de imágenes de grafiti, sin abordar cuestiones relacionadas con la legalidad, la ética o el impacto social del grafiti.

1.5 Estructura del documento

Este documento se organiza en seis secciones principales. Tras esta introducción, se discuten los trabajos relacionados en la sección 2. Posteriormente, se mencionan las herramientas y tecnologías utilizadas, así como la metodología seguida para la implementación y evaluación del sistema en las secciones 3 y 4. En la sección 5 se presentan los resultados obtenidos y se discuten sus implicaciones. Se finaliza con un resumen de los objetivos propuestos, las conclusiones alcanzadas y las líneas futuras de trabajo en la sección 6.

2 Estado de la cuestión

2.1 Análisis automático de grafiti

La literatura específica sobre la aplicación tanto de técnicas de visión por computador como de agrupamiento no supervisado al análisis de grafiti es muy escasa, reciente y de alcance acotado. Un trabajo relevante es el de Fogaça *et al.* [1], en el que se presenta un sistema encargado por el Ayuntamiento de Lisboa para clasificar imágenes de calle en grafiti ilegal, arte urbano o ausencia de grafiti y, a continuación, localizar el grafiti en la escena mediante un detector *fine-tuned* por ellos mismos. En esta misma línea, Tokuda *et al.* [3] desarrollan un sistema para cuantificar la densidad de grafitis en la ciudad de São Paulo, Brasil. Las imágenes se obtienen a través de la plataforma de Street View de Google, y el modelo de detección utilizado (Mask-RCNN, un modelo capaz de segmentar instancias de objetos en una imagen) se entrena a partir de un conjunto de datos anotado manualmente por los autores.

Estos trabajos, aunque no están alineados del todo con el planteamiento de este, establecen que el uso de aprendizaje profundo es viable sobre imágenes de grafiti captadas en condiciones no controladas.

En el ámbito del agrupamiento no supervisado, aunque varios trabajos han abordado la categorización de grafiti por tipo o estilo (como puede ser el trabajo de Taylor *et al.* [2], en el que se buscaba una rúbrica consistente para la clasificación manual de grafiti según su estilo y contenido), la gran mayoría de estos trabajos realizan esta categorización de forma manual, sin aplicar técnicas de agrupamiento automático. En esta dirección, García García *et al.* [4] proponen un método de identificación y agrupamiento basado en el análisis de los colores dominantes de cada pieza haciendo uso de K-medias sobre los espacios RGB y Lab. La aportación es relevante porque introduce el agrupamiento en el dominio, pero utiliza exclusivamente estadísticas cromáticas de la imagen y prescinde de la estructura espacial, la textura y el contenido semántico del grafiti.

La tabla 1 resume estos trabajos previos y los sitúa frente al planteamiento del presente, haciendo visible la carencia que este trabajo pretende cubrir: ninguno combina descriptores visuales profundos con técnicas estándar de agrupamiento ni acompaña la evaluación de una caracterización del coste.

Trabajo	Tarea	Datos	Características	Técnica	Métrica
Fogaça (2023)	Clasificación y detección de grafiti	Imágenes de calle (Lisboa)	CNN <i>fine-tuned</i>	Clasificación supervisada + detector	Exactitud / mAP (precisión media promedio, supervisada)
Tokuda (2019)	Cuantificación de densidad de grafitis	Google Street View (São Paulo)	Mask-RCNN	Segmentación supervisada	Precisión / recall
Taylor (2010)	Categorización por contenido	Fotografías de grafiti	Inspección visual	Categorización manual	— (cualitativa)
García García (2025)	Agrupamiento de grafitis	Imágenes de grafiti	Colores dominantes (RGB / Lab)	K-medias	Interna (no supervisada)
Este trabajo	Agrupamiento y recuperación por similitud	Banco de grafiti	<i>Embeddings</i> profundos	Reducción + <i>clustering</i> clásico (comparativo)	Interna, externa y coste computacional

Cuadro 1: Comparativa de trabajos previos sobre análisis de grafiti frente al planteamiento de este trabajo.

No se han localizado trabajos previos que combinen descriptores visuales profundos con técnicas estándar de agrupamiento para esta aplicación, ni que evalúen sistemáticamente qué componentes funcionan mejor sobre fotografías de grafitis. Esta carencia en la literatura es la que delimita el espacio en el que se sitúa el presente trabajo y motiva su planteamiento combinatorio.

2.2 Análisis de similitud y agrupamiento de imágenes

El agrupamiento de imágenes se aborda habitualmente encadenando tres etapas: extracción de una representación vectorial, reducción de dimensionalidad y agrupamiento. Los modelos de clasificación preentrenados como ResNet [5] se usan ampliamente como extractores de características, y modelos más recientes como CLIP [6] o DINOv2 [7], entrenados por supervisión lingüística o auto-supervisión,

ofrecen representaciones transferibles sin ajuste específico de dominio. La reducción de dimensionalidad previa al agrupamiento (PCA, UMAP [8]) es práctica estándar para mitigar la inestabilidad de las distancias en espacios de alta dimensión. Las familias clásicas de agrupamiento (por centroides, como K-medias; por densidad, como DBSCAN y HDBSCAN; jerárquicos o utilizando modelos probabilísticos) cubren distintos supuestos estructurales sobre los datos [9,10,11]. Para la recuperación por similitud, los índices ANN como HNSW [12] o FAISS [13] son el estándar *de facto* en bases de datos vectoriales modernas.

Estos componentes están bien caracterizados de forma individual, pero no se han localizado trabajos que los evalúen de forma combinada sobre imágenes de grafiti ni que caractericen el coste computacional de cada etapa en función del tamaño del banco. Esta carencia es la que motiva el enfoque comparativo y empírico de este trabajo.

2.3 Identificación de autoría a partir de firmas y escritura

La identificación de los grafiteros a partir de las fotografías de sus firmas es un problema muy análogo a otros ya estudiados, como pueden ser la verificación de firmas manuscritas o el reconocimiento facial. Para el caso de la identificación de autoría, Fiel y Sablatnig [14] hacen uso de las activaciones de una red convolucional entrenada para la clasificación de escritores para recuperar la identidad de un productor a partir de una muestra de su escritura. Estos vectores de características se almacenan y la consulta se resuelve por vecindad más cercana sobre los vectores guardados. Este patrón de «codificar una vez y recuperar por similitud» es precisamente el que aplica este trabajo, trasladando los descriptores profundos descritos en la Capítulo anterior al dominio específico de la autoría.

El área del reconocimiento facial, por su parte, ha experimentado un auge en los últimos años gracias a la disponibilidad de grandes conjuntos de datos etiquetados y a la mejora de las arquitecturas de redes neuronales. FaceNet [15] es un modelo que aprende a mapear imágenes de rostros a un espacio de características donde las distancias corresponden a la similitud facial. Este enfoque se basa en el aprendizaje métrico, utilizando una función de pérdida «por tripletes», que fomenta que una imagen de un individuo quede más próxima a otras imágenes del mismo individuo que a las de individuos diferentes. La identificación de autoría en grafiti puede beneficiarse de este enfoque, ya que también se trata de un problema de aprendizaje métrico en el que el número de identidades no se conoce *a priori* y las muestras por clase son escasas.

2.4 Aprendizaje profundo de clústeres y posicionamiento de este trabajo

Una línea más reciente integra la representación y el agrupamiento en un mismo proceso de entrenamiento. Min *et al.* [16] ofrecen una revisión exhaustiva de estas propuestas de *deep clustering*. Entre las más relevantes se encuentran DeepCluster [17], que reagrupa iterativamente las características de la red con K-medias y emplea las asignaciones resultantes como pseudo-etiquetas para reentrenarla, o

SCAN [18], que disocia el aprendizaje de características auto-supervisadas de la asignación de clústeres con resultados muy superiores en conjuntos de evaluación estándar.

Estas aproximaciones, sin embargo, presuponen un volumen de datos y acceso a cómputo de los que no se dispone en el escenario de este trabajo, y resultan más difíciles de interpretar y de operar. Por ese motivo, el presente trabajo se sitúa deliberadamente en el enfoque desacoplado: aprovechar las representaciones ya disponibles en modelos preentrenados, posiblemente ajustadas con una cabeza ligera específica de dominio, y combinarlas con algoritmos de *clustering* maduros y bien comprendidos. La contribución reside en evaluar de forma sistemática qué combinaciones de extractor, reducción y agrupamiento se comportan mejor sobre imágenes de grafiti.

3 Materiales y métodos

3.1 Conjunto de datos

El banco de imágenes de grafiti utilizado en este trabajo se compone de 6681 fotografías capturadas en condiciones reales en la ciudad de Salamanca. En la figura 2 se puede ver un mapa con la ubicación de las imágenes capturadas en la ciudad que contenían información de GPS. Las imágenes presentan una alta variabilidad cromática y de escenas urbanas, con condiciones de iluminación diversas. Tampoco existe un lienzo por defecto para los grafitis, sino que aparecen en muros, puertas, vallas y mobiliario urbano en general. En muchas ocasiones, y debido a la alta densidad de grafitis en ciertas zonas, una misma imagen puede contener varios grafitis distintos, lo que añade complejidad a cualquier labor de clasificación o agrupamiento.

Las fotografías comenzaron a tomarse a mediados de 2022, aunque no fue hasta el otoño de ese mismo año que se intensificó la toma de imágenes, capturando prácticamente el dataset completo en el mes de octubre de ese año. Durante el primer trimestre de 2023 se tomaron las últimas 500 fotografías de manera más dispersa, como se puede ver en la figura 3.

Todas las imágenes se tomaron con un teléfono móvil, en específico con un Samsung Galaxy S7 Edge. La resolución de las imágenes suele ser de 4032×3024 o 4032×2268 píxeles, con una definición media de 10.6 MP. No se ha aplicado ningún tipo de filtrado ni postprocesado sobre las imágenes originales más que el realizado por defecto por el dispositivo, de modo que la evaluación opera directamente sobre el material en bruto.

De manera adicional a este banco de datos, se dispone de otro conjunto de 1106 imágenes de la ciudad de Cuenca cedidas por la iniciativa StopGrafiti¹. Estas imágenes, aunque no se incluyen en el banco principal, se utilizan para el ajuste fino del detector de grafitis, así como de las cabezas de proyección ajustadas al dominio. El conjunto de StopGrafiti fue especialmente útil para esto, puesto que se trata de un banco de grafiti con autores, estilos y entornos

¹ <https://stopgraffiti.com/>

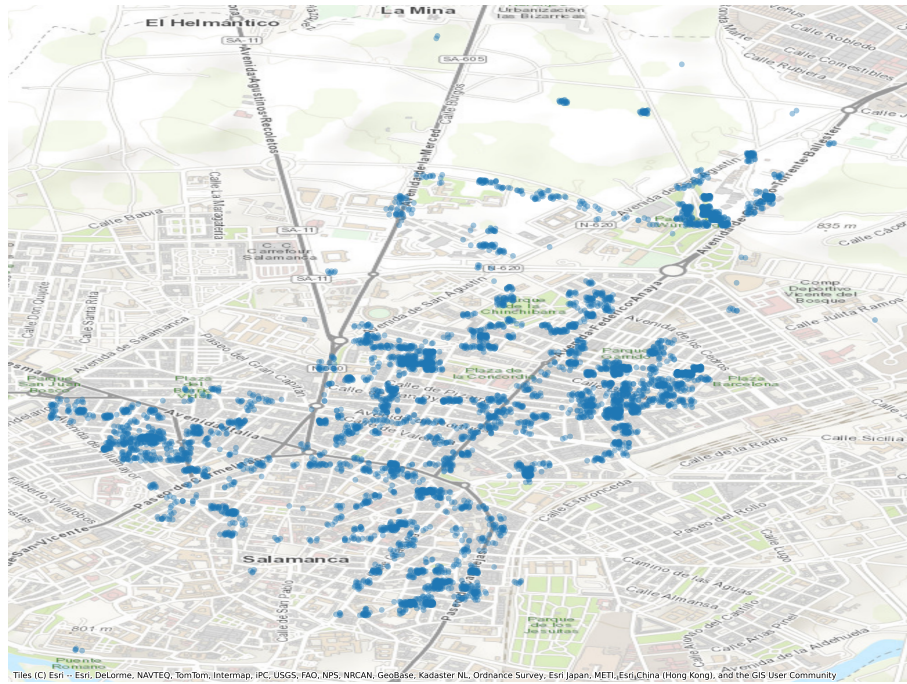


Figura 2. Ubicación de las imágenes con información de GPS en la ciudad de Salamanca.

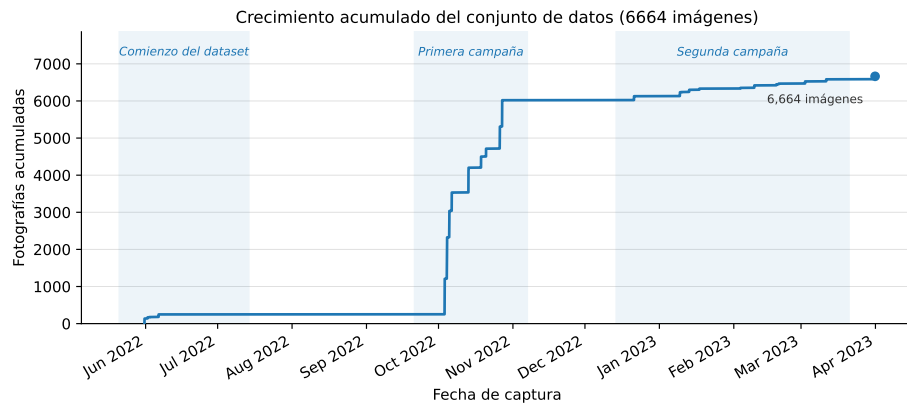


Figura 3. Progreso de toma de imágenes de grafiti a lo largo del tiempo. La gran mayoría de las imágenes se capturaron en el otoño de 2022. Durante el primer trimestre de 2023 se añadieron las últimas 500 imágenes al conjunto de datos.

distintos a los de Salamanca, lo que permite evitar el sesgo de solapamiento entre

entrenamiento y evaluación que se produciría si se empleara el banco principal para ajustar todos los modelos.

Ninguno de los dos bancos de datos cuenta con anotaciones de ningún tipo, lo que impide la evaluación supervisada directa de los modelos. Además, la existencia de varios grafitis por imagen hace que la anotación a nivel de imagen no sea suficiente para evaluar la calidad de los clústeres. Por tanto, se opta por recortar y etiquetar manualmente varios subconjuntos de grafitis, de modo que la evaluación supervisada se realice sobre estos recortes etiquetados.

Se etiquetan manualmente recortes por estilo, asignándoles una de las cuatro categorías estilísticas (*tag*, *throw-up*, *piece* y *character*) definidas e ilustradas en la tabla 2, y otros recortes por autoría. La procedencia de cada subconjunto difiere: el entrenamiento de estilo contiene recortes del banco principal, mientras que el de autoría se restringe a StopGrafiti; el reparto y su justificación se detallan al describir la evaluación extrínseca (4.3). De cualquier manera, de las 6681 imágenes de Salamanca, 265 se reservan para entrenamiento (ajuste fino del detector y de las cabezas, así como los recortes de autoría), y las 6416 ($5880 + 536$) restantes constituyen el conjunto de evaluación sin supervisión del sistema completo, del que nunca se extraen recortes de entrenamiento.

Ejemplo	Categoría	Definición
	<i>tag</i>	Firma estilizada del autor, monocroma y de trazo rápido en una sola línea; es la forma más simple y frecuente.
	<i>throw-up</i>	Nombre escrito en letras redondeadas, normalmente con no más de dos colores (contorno y relleno) y ejecución veloz.
	<i>piece</i>	Obra elaborada, multicolor y con letras complejas, sombreados y mayor inversión de tiempo.
	<i>character</i>	Figura o personaje ilustrado en lugar de letras, a menudo acompañando a otras categorías.

Cuadro 2. Categorías estilísticas etiquetadas, con un recorte de ejemplo y su definición.

La figura 4 muestra la distribución por clase de los recortes etiquetados por estilo. El reparto es desigual: las categorías *tag*, *piece* y *throw-up* concentran la mayor parte de los recortes, mientras que *character* resulta claramente minoritaria, lo que condiciona la interpretación posterior de las métricas extrínsecas por estilo.

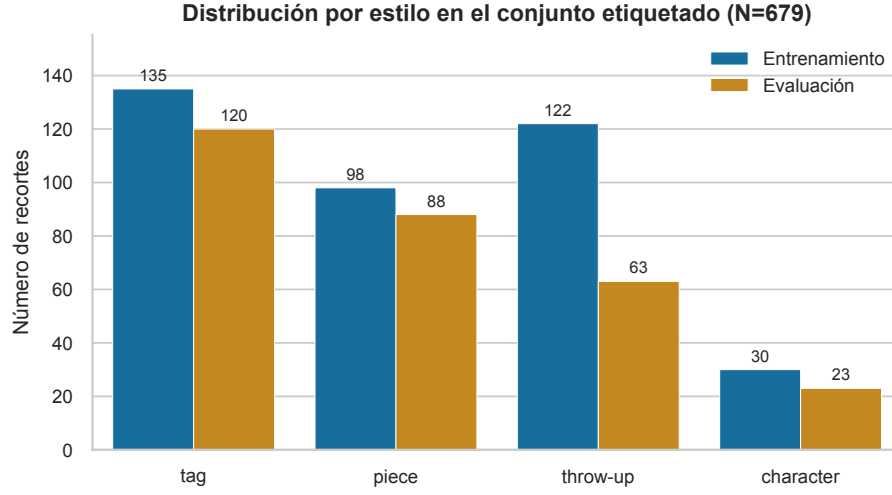


Figura 4. Distribución por clase de los recortes etiquetados por estilo, separando los subconjuntos de entrenamiento y evaluación.

Por último, se utilizan imágenes de ambos grupos para el ajuste fino del detector de grafitis, con un total de 447 imágenes completamente anotadas con las cajas de los grafitis.

La tabla 3 resume los subconjuntos etiquetados empleados, su propósito, procedencia y tamaño.

Subconjunto	Propósito	Procedencia	Tamaño
Estilo (entr.)	Cabezas de estilo	StopGrafiti + Salamanca	385
Estilo (eval.)	M. Extrínsecas	Banco completo	294
Autoría (entr.)	Cabezas de autoría	StopGrafiti	321
Autoría (eval.)	M. Extrínsecas	Salamanca	185
Detección	Ajuste del detector	Ambos bancos	447
No superv.	Agrupamiento	Salamanca (disjunto)	6416

Cuadro 3. Subconjuntos empleados (en recortes, salvo detección y no supervisión, en imágenes). Los recortes de entrenamiento nunca se solapan con los de evaluación.

3.2 Marco conceptual

Representación vectorial y métricas de distancia Como se mencionó en el estado de la cuestión, el sistema se apoya en la representación vectorial de los grafitis. Estas representaciones, conocidas como *embeddings*, se obtienen a través de modelos de extracción de características preentrenados sobre tareas de

visión generalistas, que se aplican directamente sobre las imágenes de grafiti o sobre recortes de las mismas. Para las redes de clasificación como pueden ser ResNet o VGG, se accede a esta representación eliminando las últimas capas de los modelos, que suelen corresponder a la tarea de clasificación, y tomando solamente con la representación latente previa. Otros modelos como DINOv2 o CLIP están entrenados específicamente para devolver una representación vectorial de la imagen, por lo que no es necesario modificar su arquitectura para acceder a los *embeddings*.

Teniendo una representación numérica de cada imagen, la similitud entre estas se mide mediante métricas de distancia sobre sus vectores en el espacio utilizado. Las dos más habituales son la distancia euclidiana y la distancia del coseno. Esta última depende solo de la dirección de los vectores y equivale a la euclidiana cuando los *embeddings* están normalizados a norma unitaria (normalización L_2), práctica frecuente para que la magnitud del vector no domine la comparación. Algunos modelos de este trabajo (CLIP, MobileNetV3 y las cabezas ajustadas al dominio) ya producen *embeddings* normalizados, por lo que la distancia coseno es su métrica natural; el resto usan la euclidiana. El modelo que hace uso de MobileNetV3 explícitamente normaliza sus vectores a norma unitaria para que las cabezas entrenadas en estilo y autoría (que sí que normalizan sus vectores) puedan compararse con los *backbones* de los que proceden.

Búsqueda por similitud Para la búsqueda por similitud la propuesta considera dos enfoques. El primero y más simple de ellos consiste en una búsqueda exacta sobre SQLite, que se resuelve por fuerza bruta recuperando todos los vectores almacenados y calculando la distancia a cada uno de ellos. Este método es ineficiente en cuanto a velocidad y memoria, pero garantiza la recuperación del vecino más cercano real. Por otro lado, el segundo enfoque hace uso de un índice HNSW sobre PostgreSQL, que organiza la información en una estructura de grafo jerárquico de vecinos más cercanos. Al recibir una consulta, el índice recorre la lista de vecinos más cercanos a la consulta, y a su vez los vecinos de estos, hasta encontrar el mínimo local. Este método es mucho más eficiente, aunque no garantiza la recuperación del vecino más cercano real, sino de un vecino cercano, lo que puede afectar a la calidad de los resultados.

Para evaluar esto último, al utilizar el índice HNSW también se realiza una búsqueda por fuerza bruta y se comparan los resultados obtenidos por ambos métodos, cuantificando el porcentaje de vecinos más cercanos reales recuperados por el índice. Además, cuando el dataset utilizado dispone de etiquetas se evalúa también la calidad de la búsqueda por similitud. En estos casos se calculan métricas como la precisión y *recall* a k , que cuantifican el porcentaje de elementos relevantes recuperados respecto a los k buscados ($P@k$) o al total de elementos relevantes ($R@k$). También se calculan el rango recíproco medio (MRP) que mide la posición del primer vecino relevante recuperado, y la precisión media promedio (MAP@ k) que promedia la precisión a k sobre las posiciones relevantes.

Agrupamiento no supervisado Los principales algoritmos de agrupamiento se diferencian por la manera en la que agrupan los datos. Los métodos basados en centroides, como K-medias [11], modelan cada clúster por su media y buscan minimizar la varianza intracluster. Esto los hace bastante eficientes, pero también sensibles a la forma y al solapamiento de los grupos. Por esto, estos algoritmos no suelen proporcionar los mejores resultados cuando los datos no se agrupan en formas esféricas bien separadas o cuando los distintos grupos tienen una cantidad de puntos muy desigual (como es el caso de este trabajo).

Los algoritmos basados en densidad, como DBSCAN [9], OPTICS [19] y HDBSCAN [10], definen un clúster como una región conectada de alta densidad y permiten descartar puntos como ruido. Los tres algoritmos son muy similares; siendo OPTICS y HDBSCAN extensiones de DBSCAN que resuelven algunas de sus limitaciones como pueden ser el manejo de clústeres de distintas densidades o la dificultad de ajustar el umbral de densidad. Estos métodos son especialmente adecuados para datos con formas arbitrarias y con ruido, aunque su rendimiento puede degradarse en espacios de alta dimensionalidad, lo que motiva la reducción previa.

También se utilizan otros métodos menos comunes como *affinity propagation* [20], que crea *clusters* a partir del envío de mensajes de responsabilidad entre los puntos; los métodos probabilísticos, como los modelos de mezcla gaussiana [21], que modelan los datos como una superposición de distribuciones gaussianas; los métodos espectrales [22], que utilizan la descomposición espectral de una matriz de similitud para reducir la dimensionalidad antes de aplicar un método de agrupamiento tradicional; y los métodos jerárquicos [23], que construyen un dendrograma de fusiones o divisiones sucesivas del que pueden extraerse particiones a distintos niveles de granularidad. Estos algoritmos resultan ser útiles para tratar con datos con estructuras más complejas, aunque su coste computacional suele ser mayor que el de los métodos más simples (especialmente *affinity propagation* y los métodos basados en modelos probabilísticos).

La principal dificultad en un escenario sin etiquetas es la selección del número de clústeres. K-medias y los modelos de mezcla requieren fijarlo *a priori*, y se recurre a heurísticas como el método del codo sobre la inercia o el criterio bayesiano de información (BIC). Los algoritmos basados en densidad y los jerárquicos lo evitan en su formulación, pero a cambio introducen otros hiperparámetros (umbrales de densidad, criterios de corte) cuyo ajuste también carece de un óptimo evidente sin verdad fundamental.

Reducción de dimensionalidad Los *embeddings* producidos por las redes utilizadas en este trabajo contienen entre 128 y 4096 dimensiones. A medida que la dimensionalidad crece, las distancias entre puntos tienden a igualarse y perder poder discriminativo, fenómeno conocido como la maldición de la dimensionalidad [24]. Esto degrada especialmente los algoritmos basados en densidad y motiva aplicar una etapa previa de reducción. Las técnicas lineales, como PCA [25], proyectan los datos sobre los ejes que generan la máxima varianza. Esto los hace eficientes, pero los limita a capturar relaciones lineales entre las

características. Kernel PCA [26] resuelve este problema calculando esta varianza en un espacio de características no lineal, permitiendo capturar relaciones más complejas a costa de un mayor coste computacional.

Otras técnicas no lineales, como Isomap [27] o UMAP [8] buscan preservar relaciones de vecindad en el espacio original. Isomap construye un grafo de vecinos más cercanos sobre el espacio original y posteriormente busca una representación de menor dimensionalidad que preserve estas relaciones de vecindad, lo que lo convierte en un algoritmo especialmente costoso a medida que el tamaño del conjunto crece. UMAP, por su parte, asigna a cada enlace entre puntos una probabilidad de conexión basada en la distancia entre estos, y luego busca una representación de menor dimensionalidad que respete estas probabilidades a través de una serie de operaciones de atracción–repulsión entre los puntos.

Métricas de evaluación La calidad de los clústeres resultantes se valora con dos familias de métricas. Sobre el banco principal sin etiquetar se recurre a métricas internas, que cuantifican la cohesión intracluster y la separación intercluster directamente sobre los *embeddings*. Se emplean el coeficiente de silueta [28], el índice de Calinski–Harabász [29] y el índice de Davies–Bouldin [30]. El primero de estos mide, para cada punto, la diferencia entre la distancia media a los puntos del *cluster* más cercano y la distancia media a los puntos de su propio *cluster*, normalizada por la mayor de las dos. Este favorece a los agrupamientos con clústeres compactos y bien separados. El índice de Calinski–Harabász fomenta clústeres muy compactos y separados entre sí al medir la razón entre la varianza intercluster y la varianza intracluster, penalizando los agrupamientos con clústeres dispersos o con solapamiento. Por último, el índice de Davies–Bouldin realiza la evaluación opuesta: para cada *cluster* calcula la razón entre la suma de la dispersión intracluster y la distancia intercluster a su vecino más cercano, y devuelve la media de estas razones. Esto penaliza el peor caso de cada agrupamiento, favoreciendo aquellos con clústeres bien separados y con baja dispersión intracluster.

Puesto que algunos de los algoritmos admiten la clasificación de puntos como ruido, estas métricas se complementan con tres indicadores estructurales: el número de *clusters*, la proporción de puntos clasificados como ruido y el coeficiente de variación del tamaño de los clústeres. Haciendo uso de estas métricas, se puede identificar si un valor anormalmente alto en las métricas de calidad se debe a un muy buen agrupamiento o a un agrupamiento en el que los puntos más difíciles de clasificar se han descartado como ruido o se han agrupado en un único clúster muy grande.

Adicionalmente, sobre los recortes de evaluación etiquetados por estilo o autoría se calculan métricas extrínsecas que sí utilizan la verdad fundamental disponible: el índice de Rand ajustado (ARI), la información mutua normalizada (NMI) y la F1 por pares, que constituyen el complemento más sólido a las métricas internas en el plano de calidad.

3.3 Catálogo de componentes

Esta subsección cataloga los componentes concretos integrados en el sistema, que constituyen los materiales sobre los que opera la evaluación. Su combinación parametrizada para definir cada experimento se describe en 4.3.

Modelos de extracción de características Todos los modelos se utilizan congelados, sin reentrenar sus pesos. En algunos casos se retiran las últimas capas que corresponden a la tarea de clasificación para acceder a la representación latente previa (ResNet50, VGG16 e InceptionV3, por ejemplo). Para el resto de modelos se utiliza la propia API de extracción de características, que ya devuelve la representación latente sin necesidad de modificar la arquitectura (DINOv2, CLIP y YOLO). La tabla 4 resume las opciones disponibles, indicando arquitectura, dimensionalidad de salida y preentrenamiento.

Modelo	Arquitectura	Dim.	Preentrenamiento
ResNet50 [5]	CNN residual	2048	ImageNet
VGG16 [31]	CNN profunda	4096	ImageNet
InceptionV3 [32]	CNN multi-rama	2048	ImageNet
MobileNetV3 [33]	CNN ligera	1280	ImageNet
DINOv2 ViT-S/14 [7]	<i>Transformer</i> [34]	384	Autosupervisado (LVD-142M)
CLIP ViT-B/32 [6]	<i>Transformer</i>	512	Imagen-texto (WIT)
YOLO11 (n/m) [35]	Detector <i>one-stage</i>	256–512	COCO

Cuadro 4: Modelos de extracción de características integrados.

Algoritmos de agrupamiento El sistema cubre los principales paradigmas de agrupamiento descritos en la sección 3.2. Todas las implementaciones exponen una interfaz común y se configuran por hiperparámetros declarados en la configuración del *benchmark*. Estas implementaciones no se han implementado desde cero, sino que se hace uso de las implementaciones disponibles en scikit-learn [36]. La tabla 5 recoge las opciones implementadas con sus hiperparámetros clave.

La columna «Auto. k » indica qué algoritmos pueden estimar su hiperparámetro principal cuando este no se declara en la configuración; el procedimiento empleado en cada caso y su papel en el protocolo comparativo se detallan en 4.3.

Algoritmo	Hiperparámetros principales	Auto. k
K-medias [11]	$n_clusters$	sí
DBSCAN [9]	$\epsilon, min_samples$	sí (ϵ)
HDBSCAN [10]	$min_cluster_size, max_cluster_size$	no requiere
OPTICS [19]	$min_samples, max_eps, metric$	no requiere
Aglomerativo [23]	$n_clusters, linkage$	no

Espectral [22,37]	<i>n_clusters</i> , afinidad	no
GMM [21]	<i>n_components</i> , covarianza	sí
<i>Affinity Propagation</i> [20]	<i>damping</i> , <i>preference</i>	no requiere

Cuadro 5: Algoritmos de agrupamiento implementados.

Técnicas de reducción de dimensionalidad La etapa de reducción es opcional, siendo la implementación por defecto una identidad que no aplica ninguna transformación, permitiendo comparar el efecto de añadir esta etapa. La tabla 6 resume las opciones disponibles.

Técnica	Tipo	Parámetros principales
Identity	—	—
PCA [25]	Lineal	<i>n_components</i>
Kernel PCA [26]	No lineal	<i>n_components</i> , <i>kernel</i>
Isomap [27]	No lineal	<i>n_components</i> , <i>n_neighbors</i>
UMAP [8]	No lineal	<i>n_components</i> , <i>n_neighbors</i> , <i>min_dist</i>

Cuadro 6: Técnicas de reducción de dimensionalidad implementadas.

Sistemas de almacenamiento La persistencia de los *embeddings* y los metadatos asociados se abstrae tras una interfaz común, de la que se han implementado dos *backends* con características operativas distintas. El primero, basado en SQLite, no requiere servicios externos, almacena los vectores como *blobs* y realiza la búsqueda por similitud mediante un recorrido lineal; resulta apropiado para despliegues ligeros y como referencia de cálculo exacto frente a alternativas aproximadas. El segundo se apoya en la extensión *pgvector* de PostgreSQL y delega la búsqueda al motor de la base de datos. Por defecto la consulta es un recorrido exacto; opcionalmente puede habilitarse un índice aproximado HNSW [12] sobre los vectores, lo que permite contrastar el coste de la búsqueda exacta frente a la aproximada. El índice HNSW de *pgvector* impone un límite de 2000 dimensiones sobre el tipo *vector*, por lo que no es aplicable a los modelos cuya salida lo supera (ResNet50 e InceptionV3, con 2048, y VGG16, con 4096); para estos casos debe recurrirse a una reducción de dimensionalidad previa al almacenamiento o al *backend* SQLite con búsqueda exacta. Ambos *backends* comparten el esquema lógico y la persistencia de metadatos, lo que permite cambiar de uno a otro sin modificar el resto del *pipeline*.

3.4 Extensiones específicas para grafiti

Segmentador YOLO La etapa de segmentación localiza los grafitis dentro de la fotografía para que el extractor de características opere sobre los recortes

correspondientes en lugar de la escena completa. Se han implementado dos variantes. La primera de ellas no aplica ningún tipo de segmentación, de modo que el extractor de características se aplica sobre la fotografía completa. La segunda, utiliza un modelo de YOLO [35] en su versión oficial o ajustado sobre el conjunto de grafiti para detectar las cajas que encierran cada grafiti, y recorta la imagen en consecuencia. Además, este proceso de recorte fusiona las cajas de detección solapadas para evitar duplicados y aplica un margen configurable alrededor de cada caja antes del recorte. El resultado es un conjunto de sub-imágenes que se procesan de forma independiente y que pueden producir varios vectores por fotografía original.

Además de la fusión por solapamiento estándar, el segmentador incorpora un criterio de proximidad que une cajas adyacentes cuya separación no supera un umbral, consolidando grafitis contiguos en una misma pared. Una salvaguarda rechaza fusiones cuya caja unión supere una fracción del área de la imagen, evitando que cadenas de fusiones por contacto generen un recorte que cubra la escena completa. Tras la fusión, la lista de detecciones puede truncarse a las K de mayor confianza, mitigando la fragmentación en imágenes con muchas detecciones dispersas de baja calidad. Por último, cuando el detector no emite ninguna detección, se puede optar por emitir la imagen completa como caja única o descartar la imagen del análisis.

Para evaluar la calidad de este segmentador se hace uso de las métricas de detección estándar: precisión, *recall* y la precisión media promedio (mAP) a distintos umbrales de intersección sobre unión (IoU). La IoU mide el solapamiento entre la caja predicha y la caja correcta como el cociente entre el área de su intersección y el área de su unión, de modo que vale 1 cuando coinciden por completo y 0 cuando son disjuntas. Las dos primeras funcionan de la misma manera que la búsqueda por similitud, cuantificando el porcentaje de detecciones relevantes recuperadas respecto al total de detecciones emitidas (precisión) o al total de elementos relevantes (*recall*). La mAP agrega ambas en una sola cifra: para un umbral de IoU fijo se calcula la precisión media (*average precision*, AP), que corresponde al área bajo la curva precisión-*recall*, y la mAP es la media de ese AP sobre todas las clases. Una caja de detección se considera correcta cuando su IoU con la caja correcta supera el umbral fijado; mAP@0.5, por ejemplo, emplea un umbral del 50%. La variante mAP@0.5-0.95 promedia además la mAP sobre umbrales de IoU entre el 50% y el 95% en pasos del 5%, lo que proporciona una evaluación más completa de la calidad de las detecciones.

El detector se ajusta sobre el subconjunto anotado de ambos bancos descrito en 3.1, disjunto del banco de evaluación, lo que evita que las métricas reportadas sobre las configuraciones con segmentador queden sesgadas por el solapamiento entre entrenamiento y evaluación. El procedimiento de ajuste fino se detalla en 4.4.

Cabezas de proyección ajustadas al dominio Los modelos de fundación generalistas producen representaciones transferibles, pero no necesariamente óptimas para el dominio del grafiti. Para acortar esa distancia se ha optado por un

esquema de *transfer learning* [38] en el que la representación base se mantiene congelada y solo se entrena una cabeza ligera, siguiendo la metodología de *fine-tuning* de ULMFiT [39]. La cabeza consiste en una secuencia Lineal $\rightarrow$ ReLU $\rightarrow$ Lineal que proyecta los *embeddings* base a un espacio adaptado al dominio. Además, su salida se normaliza a norma unitaria (L_2), de modo que el *embedding* producido sea directamente comparable mediante distancia coseno (Figura 5).

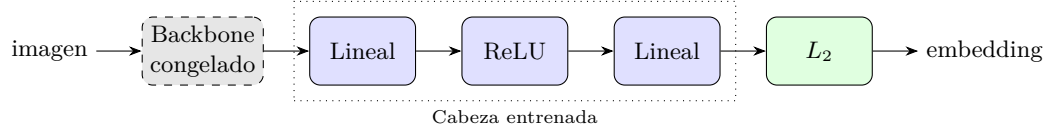


Figura 5. Cabeza de proyección ajustada a grafiti. El *backbone* permanece congelado y solo se entrena una secuencia ligera de capas para luego normalizar la salida a norma unitaria.

Se han entrenado dos tipos de cabezas (de autoría y de estilo) sobre dos arquitecturas base (MobileNetV3 y DINOv2). Las cabezas de autoría buscan acercar en el espacio latente los grafitis de un mismo autor y separar los de autores distintos; las de estilo organizan el espacio según las cuatro categorías estilísticas (*tag*, *throw-up*, *piece* y *character*). Las cuatro cabezas resultantes se entrenan exclusivamente sobre los subconjuntos anotados descritos en 3.1 (detalles sobre el entrenamiento en 4.4) y se añaden al catálogo de modelos de *embedding*, proyectando los *embeddings* base a un espacio de 128 dimensiones (DINOv2 $384 \rightarrow 128$, MobileNetV3 $1280 \rightarrow 128$).

3.5 Entorno de ejecución

Los experimentos se realizaron en el ordenador personal del autor: un portátil con CPU AMD Ryzen 7 5800H, GPU NVIDIA GeForce GTX 1650 y 16 GB de RAM DDR4. El entorno de ejecución se basa en Python 3.14 gestionado con *uv*, empleando como dependencias nucleares PyTorch (2.10.0), *open-clip-torch*, *ultralytics*, *scikit-learn*, *umap-learn*, *SQLAlchemy* con *pgvector*, *numpy*, *pandas*, *matplotlib*, *seaborn*, *psutil* y *tqdm*. Salvo PyTorch, fijado de forma exacta, las versiones corresponden al requisito mínimo declarado en el gestor de dependencias. La tabla 7 recoge las versiones de las dependencias principales.

4 Metodología

4.1 Visión general del flujo de trabajo

El sistema implementado procesa cada fotografía en cuatro etapas operativas: segmentación de los grafitis contenidos en la imagen, extracción de un vector de características por recorte, persistencia del vector junto a sus metadatos en un

Dependencia	Versión	Dependencia	Versión
Python	3.14	<i>scikit-learn</i>	1.8.0
PyTorch	2.10.0	<i>umap-learn</i>	0.5.12
<i>open-clip-torch</i>	3.3.0	<i>SQLAlchemy</i>	2.0.49
<i>transformers</i>	5.7.0	<i>pgvector</i>	0.4.2
<i>ultralitics</i>	8.4.26	<i>numpy</i>	2.4.3
<i>pandas</i>	3.0.1	<i>matplotlib</i>	3.10.8
<i>seaborn</i>	0.13.0	<i>psutil</i>	6.1.1
<i>tqdm</i>	4.67.3	<i>psycopg2</i>	2.9.11

Cuadro 7. Versiones de las dependencias principales del proyecto. Salvo PyTorch (fijado de forma exacta), corresponden al requisito mínimo declarado.

almacenamiento estructurado y, en un segundo tiempo, recuperación masiva del conjunto almacenado para reducirlo y agruparlo. La separación explícita entre la fase de ingesta y la fase de agrupamiento es intencionada. Por un lado, refleja el escenario de uso previsto (un banco que crece de forma incremental sobre el que se ejecutan sesiones de análisis puntuales), y por otro permite aislar el coste computacional de cada etapa al medir (figura 6).

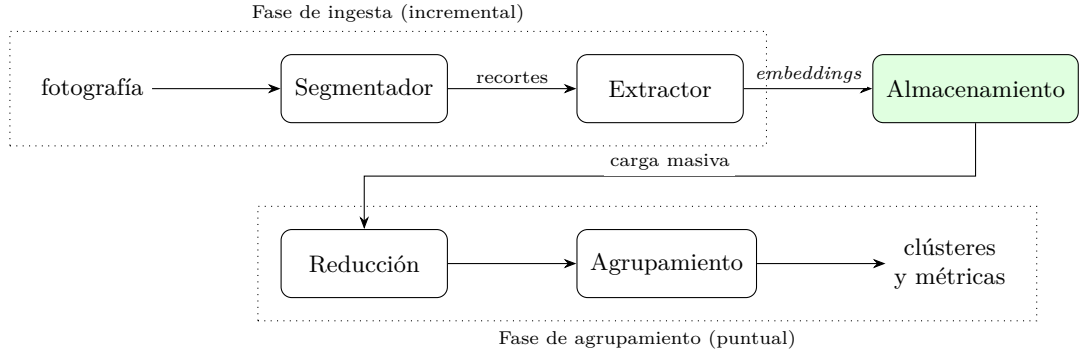


Figura 6. Flujo de trabajo en dos fases. El almacenamiento es el punto de acoplamiento entre la ingesta incremental y la fase de agrupamiento puntual.

Cada una de las cinco etapas es un componente intercambiable tras una interfaz común, catalogados en 3.3. Este desacoplamiento es el que habilita la evaluación combinatoria que forma el trabajo: cada combinación define una configuración independiente y reproducible cuyo coste y calidad pueden compararse con cualquier otra.

4.2 Pipeline operativo

Ingesta y generación de *embeddings* La ingesta de cada fotografía sigue la siguiente secuencia de pasos:

1. La imagen se carga desde disco y se proporciona al segmentador.
2. El segmentador consume la imagen y devuelve una lista con cajas delimitadoras de los grafitis presentes en la imagen. En el caso de que no se utilice segmentador, se devuelve una única caja que abarca toda la imagen, de modo que el sistema se comporta como un agrupador de imágenes completas. Este comportamiento de respaldo también se puede habilitar o deshabilitar para los modelos de detección de grafitis, de modo que las imágenes sin detección se emitan como recortes de escena completa o se descarten, respectivamente.
3. Las cajas detectadas se fusionan según criterios de solapamiento y proximidad configurables, con salvaguardas para evitar mega-cajas; opcionalmente se trunca la lista a las K de mayor confianza. Si se configura un margen no nulo, las cajas supervivientes se expanden para incluir contexto visual alrededor del grafiti. Los parámetros que gobiernan este proceso se describen en 3.4.
4. Cada grafiti se recorta sobre la imagen original y el recorte se entrega al modelo de extracción de características, que produce un vector de características de dimensión fija.
5. El vector resultante, junto con la referencia a la fotografía original y las coordenadas del recorte se almacenan en el sistema elegido.

La figura 7 ilustra el recorrido completo sobre una fotografía real: las cajas que emite el detector, su fusión por solapamiento, la expansión por el margen configurado y los recortes finales que se entregan al extractor.

Recuperación, reducción y agrupamiento Una vez completada la ingesta, la fase de agrupamiento opera sobre el contenido completo del almacenamiento. En primer lugar, se recuperan todos los *embeddings* junto con sus metadatos en una única consulta. A continuación, si la configuración declara una reducción, se aplica sobre cada vector de características el método de reducción seleccionado, obteniendo un nuevo conjunto de vectores. Finalmente, el algoritmo de agrupamiento se ajusta sobre el conjunto reducido, produciendo una etiqueta de clúster por cada vector. A partir de esta información se calculan las métricas de calidad intrínsecas y extrínsecas, y se conserva el mapeo entre identificadores de clúster y listas de recortes para la generación de visualizaciones interactivas. Esta secuencia de pasos se ejecuta además varias veces para cada configuración para obtener estadísticas sobre la variabilidad de las métricas de coste y calidad.

4.3 Procedimiento experimental

La evaluación se articula en dos ejes. El primero, y principal, es el rendimiento computacional, cuantificado mediante los tiempos absolutos de cada etapa del



Figura 7. Etapas de la segmentación sobre una fotografía del banco, siguiendo el flujo de las flechas: la fotografía original; las detecciones del modelo YOLO ajustado; las cajas resultantes tras fusionar las solapadas; las cajas expandidas con el margen de contexto; y los recortes finales que se entregan al extractor.

pipeline (preparación, ingesta, reducción, agrupamiento y búsqueda por similitud), el *throughput* de ingesta en imágenes por segundo, la latencia de búsqueda por consulta y la escalabilidad de estas magnitudes con el tamaño del banco; todas ellas son métricas objetivas e independientes de la existencia de etiquetas. El segundo eje, secundario, es la calidad de los clústeres resultantes, evaluada con las métricas internas y extrínsecas definidas en 3.2.

Definición del espacio de configuraciones Cada experimento queda descrito por un fichero de configuración en formato JSON. El fichero contiene una lista de experimentos. Cada experimento declara un identificador único y cinco componentes con la forma `{type, params}`: `storage`, `embedding`, `segmenter`, `reduction` y `clustering`, además de un bloque opcional `similarity_search`. Esta forma declarativa garantiza que toda la información necesaria para reproducir un experimento queda en un único fichero versionable.

Para barrer sistemáticamente el espacio combinatorio se utiliza un fichero auxiliar en el que cada componente se sustituye por una lista de alternativas. Un *script* expande este fichero mediante producto cartesiano, materializando un experimento en el archivo final por cada combinación válida.

Protocolo de escalabilidad La caracterización del coste computacional en función del tamaño del banco constituye uno de los ejes centrales del trabajo. Para obtenerla, se ejecuta cada configuración seleccionada sobre una sucesión creciente de tamaños N controlada mediante el parámetro `--limit`. La selección

de tamaños es reducida en la mayoría de experimentos para evitar que el coste total de la evaluación se dispare, pero se realiza un barrido completo sobre un rango de tamaños lo suficientemente amplio como para identificar tendencias de crecimiento.

El muestreo de las imágenes para cada N es determinista: las fotografías se ordenan por su nombre de fichero y se toman las primeras N . Para cada tamaño se obtienen las métricas temporales descritas en 4.3, lo que permite caracterizar el crecimiento de cada etapa por separado y, a partir de ahí, estimar empíricamente el orden de crecimiento de la función de coste respecto a N para cada etapa.

Reutilización de almacenamientos La ingesta es, en la mayoría de ocasiones, la etapa más costosa del *pipeline*. Para evitar repetirla, se crea un identificador para cada almacenamiento a partir de la configuración de ingesta (segmentador, modelo de representación y parámetros asociados) que luego es usado para situar la base de datos en un directorio específico (o nombrar la tabla en el caso de PostgreSQL). Cuando el sistema detecta que el almacenamiento ya existe y no se ha declarado explícitamente que se limpie (`clear_storage: true`), la fase de ingesta se omite y se reutiliza el almacenamiento existente. Las métricas de ingesta se almacenan en la tabla de metadatos del almacenamiento para que, en casos como estos, se reporten los tiempos reales de ingesta en lugar de los tiempos de caché, que no reflejarían el coste real de la etapa.

Instrumentación y cronometría El *runner* de evaluación mide cuatro etapas: ingesta, reducción, agrupamiento y búsqueda por similitud. Cada una se envuelve en un gestor de contexto (`profile_stage`) que registra los instantes de entrada y salida con `time.perf_counter`, un reloj monotónico de alta resolución. La diferencia entre ambos instantes es el tiempo de pared (*wall time*) de la etapa, en segundos y con resolución de microsegundos. La preparación de componentes y la limpieza del almacenamiento quedan fuera del gestor de contexto y no se contabilizan.

Para acotar la variabilidad entre ejecuciones, las etapas de reducción y agrupamiento se repiten K veces dentro del mismo proceso, con las semillas $0, 1, \dots, K-1$. La semilla afecta a la reducción estocástica (UMAP) y a los agrupamientos estocásticos (K-medias, GMM, *spectral clustering* y *affinity propagation*); los algoritmos deterministas (DBSCAN, OPTICS, HDBSCAN y aglomerativo) la reciben por la interfaz común pero la ignoran. Cuando la reducción y el agrupamiento son ambos deterministas, las repeticiones serían idénticas, así que se fuerza $K = 1$; esas configuraciones, en consecuencia, no aportan barras de error.

Cuando $K \geq 2$, la primera repetición ($i = 0$) se descarta porque absorbe los efectos de inicialización (compilación *just-in-time* de operadores y poblamiento de cachés del sistema operativo); sobre las $K - 1$ restantes se reportan la media aritmética y la desviación típica de cada métrica de coste. La calidad del agrupamiento, en cambio, se evalúa sobre las K repeticiones completas: no

la afectan los costes de inicialización y así se cuantifica la sensibilidad de las métricas intrínsecas al muestreo aleatorio.

La ingesta y la búsqueda por similitud se miden una sola vez por ejecución. La ingesta es determinista y, además, la reutilización de almacenamiento (4.3) exige una cronometría única; la búsqueda por similitud es barata y su variabilidad entre configuraciones se observa directamente a lo largo del barrido de tamaños. Los experimentos se ejecutan sobre un sistema sin otras cargas significativas (3.5) para minimizar interferencias externas.

Selección automática de hiperparámetros Salvo que se desee evaluar explícitamente el efecto de fijar un hiperparámetro a un valor concreto, todas las configuraciones del *benchmark* emplean por defecto la estimación automática del hiperparámetro principal de K-medias, DBSCAN y los modelos de mezcla gaussiana, mediante el método del codo, la curva de k -distancias y el criterio bayesiano de información respectivamente. En los tres casos las semillas aleatorias se fijan para que el resultado sea reproducible.

La elección de estos algoritmos también atiende a su coste computacional: K-medias cuenta con variantes escalables como la versión *mini-batch* [40], mientras que los métodos basados en densidad presentan un crecimiento superlineal con el tamaño del banco. En cualquier caso, el rendimiento en calidad depende fuertemente del conjunto de datos [41,42], lo que refuerza el enfoque comparativo adoptado en este trabajo.

Cuando una configuración no fija `n_clusters` y emplea varias repeticiones, el número de clústeres autodetectado en la primera repetición se propaga al resto, de modo que las repeticiones varían únicamente por la semilla y las métricas agregadas describen una sola familia de particiones en lugar de mezclar particiones con distinto número de clústeres. El número de clústeres obtenido en cada repetición se registra para poder verificar que la propagación se mantiene.

Protocolo de búsqueda por similitud La medición del coste de búsqueda por similitud se realiza sobre los *backends* de almacenamiento descritos anteriormente. Tras realizarse la ingesta, se seleccionan n *embeddings* de manera aleatoria (pero reproducible) y se buscan los k vecinos más próximos a cada uno de ellos, midiendo el tiempo total de las consultas. La distancia utilizada para la búsqueda puede ser tanto euclidiana como coseno, y el número de vecinos también es configurable.

El tiempo total de las consultas se mide con el mismo procedimiento que el resto de etapas. De la misma forma que en el almacenamiento, la búsqueda por similitud es determinista una vez fijada la semilla que determina las entradas que buscar, así que no se repite varias veces como la reducción y el agrupamiento. El coste de esta etapa se espera que crezca como $O(n^2)$ para el recorrido lineal de `SQLiteStorage` y como $O(n \log n)$ para el índice HNSW de `PostgreSQLStorage`.

De esta etapa también se reporta la distancia media a los k vecinos recuperados, promediada sobre todas las consultas. Si el *backend* dispone de índice ANN (como es el caso de `PostgreSQL` con HNSW), también se reporta el $R@k$ de sus

vecinos frente a una búsqueda exacta por fuerza bruta, que cuantifica la pérdida de exactitud del índice aproximado y es nulo en los *backends* exactos.

Evaluación intrínseca de los agrupamientos Las métricas intrínsecas definidas en 3.2 se calculan de manera operativa sobre los *embeddings* originales en su espacio de partida, no sobre los *embeddings* reducidos utilizados por el agrupamiento. Esto se hace porque el objetivo de estas métricas es evaluar la calidad del agrupamiento de los datos originales, no la habilidad del reductor para deformar el espacio de modo que los clústeres aparezcan más compactos o separados.

Los algoritmos basados en densidad (DBSCAN, OPTICS y HDBSCAN) pueden asignar la etiqueta -1 a puntos considerados ruido. Estos puntos se excluyen del cálculo del coeficiente de silueta y de los índices de Calinski–Harabász y Davies–Bouldin, lo que evita penalizar a los algoritmos que identifican y reportan ruido. Por otro lado, un agrupamiento que rechaza una gran fracción de puntos como ruido podría quedar artificialmente premiado por agrupar solo los puntos muy cercanos entre sí; por ello estas métricas se complementan con la proporción de puntos etiquetados como ruido y la variedad de tamaños de los clústeres resultantes.

Evaluación extrínseca sobre los subconjuntos etiquetados La evaluación supervisada hace uso de las métricas descritas anteriormente (sección 3.2) y opera sobre dos conjuntos de recortes etiquetados, uno por cada tarea supervisada del trabajo: evaluación por estilo y evaluación por autor. En ambos casos, se utilizan los recortes directamente como imágenes a evaluar en vez de aportar la imagen original y utilizar un segmentador para extraer los recortes de evaluación, porque el objetivo es medir la calidad del espacio de representación para agrupar grafitis, no la capacidad de un sistema completo. Tampoco se utilizan imágenes completas, puesto que muchas de estas contienen varios grafitis de estilos y autores distintos, haciendo difícil atribuir una etiqueta de estilo o autor a la imagen completa y complicando la interpretación de las métricas.

Ambas evaluaciones comparten métricas y procedimiento, pero responden a preguntas distintas, y esa diferencia justifica el reparto asimétrico de las anotaciones entre entrenamiento y evaluación. Las etiquetas de estilo definen una tarea de clasificación cerrada sobre cuatro categorías fijas, así que basta con que no se solapen las fotografías entre entrenamiento y evaluación. La diversidad estilística se cubre mejor agregando recortes de Salamanca y de Cuenca (el banco StopGrafiti) al entrenamiento, y el conjunto de evaluación, extraído del banco principal de Salamanca, es disjunto a nivel de fotografía. La evaluación por estilo mide así el grado en que el espacio de representación agrupa grafitis estilísticamente homogéneos.

Las etiquetas de autor, en cambio, definen un problema abierto de aprendizaje de métricas: en el uso real del sistema, los autores de los grafitis a agrupar son en su mayoría desconocidos para el modelo en el momento de la inferencia.

Si las cabezas de autoría se entrenasen sobre los mismos identificadores de autor que luego aparecen en la evaluación (aun utilizando fotografías distintas), las métricas medirían la capacidad de reidentificar autores ya vistos, no la de construir un espacio en el que autores nuevos queden separados. Para evitar este sesgo se sigue la convención habitual de la verificación de identidad (Face-Net [15] y la literatura de reidentificación de personas): el conjunto de evaluación contiene únicamente identificadores de autor disjuntos de los vistos durante el entrenamiento.

Esta restricción se cumple entrenando las cabezas de autoría exclusivamente con los recortes de Cuenca y evaluándolas sobre los de Salamanca. Dado que ambas ciudades están geográficamente separadas, es razonable asumir que los conjuntos de autores no se solapan. Esto, aunque no se ha verificado formalmente, se ha comprobado de manera informal al realizar la segmentación y anotación de los recortes, y no se ha encontrado ningún caso de un mismo autor operando en ambas ciudades. Esta elección introduce además un cambio de dominio entre ciudades, que deberá tenerse en cuenta al interpretar los resultados de la evaluación por autor (5). La asimetría entre entrenamiento y evaluación es, por tanto, intrínseca al problema de aprendizaje de métricas.

Las métricas ARI, NMI y F1 por pares se aplican directamente sobre los *clusters* obtenidos y las etiquetas de estilo y autoría, sin resolver explícitamente el problema de asignación clúster-clase: Las tres son invariantes ante permutaciones de etiquetas y comparan las relaciones de coagrupamiento entre pares de puntos. Para los algoritmos que pueden etiquetar puntos como ruido (-1) se reportan dos lecturas de cada métrica. En la lectura por defecto los puntos de ruido se tratan como un único clúster adicional, de modo que influyen en la métrica en lugar de ignorarse silenciosamente. En la lectura complementaria (`*_no_noise`) esas instancias se descartan antes de puntuar, de forma que los algoritmos de densidad y los particionales se comparan sobre el subconjunto que cada uno realmente agrupa. Esta doble lectura es asimétrica respecto al tratamiento del ruido en las métricas intrínsecas, donde el ruido siempre se excluye y se complementan con valores como el porcentaje de ruido.

Para evaluar la calidad de la búsqueda por similitud se emplean también las etiquetas disponibles según la evaluación: las métricas $P@k$, $MAP@k$, MRR y $R@k$ definidas en 3.2 se calculan por consulta y se promedian sobre todas las consultas realizadas para obtener una media global de la calidad de la búsqueda por similitud. Como $R@k$ acota su denominador al máximo de vecinos relevantes recuperables en k posiciones ($\min(k, |clase| - 1)$), cuando las clases son grandes frente a k (como en la evaluación de estilo, con cuatro clases densas) ese denominador se satura en k y $R@k$ coincide numéricamente con $P@k$. En ese caso se omite de las tablas por redundancia y solo se incluye cuando ambas divergen.

Salida y agregación de resultados Cada lote de *benchmarks* produce un único fichero JSON con dos secciones: un bloque con información sobre el entorno de ejecución (versión de Python, librerías principales, *hardware* utilizado), y otro bloque con una entrada por configuración que contiene todas las métricas

y tiempos capturados. Para las etapas repetidas se incluyen tanto los valores medios como las desviaciones típicas y el número de muestras agregadas en cada caso, de manera que el análisis posterior pueda distinguir diferencias dentro del ruido de medición.

4.4 Entrenamiento de modelos auxiliares

Detector de grafiti El detector se entrena a partir del subconjunto anotado descrito en 3.1, cuyas 447 fotografías de ambos bancos contienen aproximadamente un total de 650 cajas anotadas. Puesto que se trata de un número bastante reducido, se realiza una aumentación de datos bastante agresiva aplicando diversas transformaciones sobre las imágenes y sus cajas correspondientes. El entrenamiento parte de los pesos preentrenados sobre COCO y solo ajusta sus parámetros para la única clase de interés, «graffiti».

Se utilizan los hiperparámetros por defecto de la implementación de referencia, salvo el número de épocas, el tamaño de *batch* y la resolución de entrada, que se fijan para encajar en la memoria de la GPU descrita en 3.5. El optimizador, las *augmentations* (jitter HSV, escalado, volteos horizontales y verticales, cambios de perspectiva, rotaciones, traslaciones, oclusiones parciales de partes de la imagen y *mosaic*) y la *learning rate* con *warm-up* y el descenso por defecto del *framework* (lineal) se mantienen en sus valores por defecto. El entrenamiento incorpora parada temprana cuando la métrica mAP@0.5 sobre validación no mejora durante un número fijo de épocas.

Cabezas de proyección Las cuatro cabezas (autoría y estilo sobre MobileNetV3 y DINOv2) se entrenan sobre la totalidad del subconjunto anotado, sin reservar una partición de validación. Los *backbones* se mantienen estrictamente congelados durante todo el proceso y solo se actualizan los parámetros de la cabeza, según el esquema de *transfer learning* descrito en 3.4.

Las cabezas de autoría se entrenan con la pérdida de tripletes en su variante *batch-hard* [43] con margen 0.3, optimizador Adam con *learning rate* 10^{-4} y 20 épocas, sin parada temprana: se conservan los pesos de la última época. Cada *batch* se construye mediante un muestreador PK que selecciona 8 autores y 4 recortes por autor (tamaño de *batch* 32), de modo que cada ancla disponga de varios pares positivos dentro del mismo *batch*. A continuación, y dada la relevancia del muestreo de negativos [44], para cada ancla se forma el triplete con su positivo más difícil (el recorte del mismo autor más alejado en el espacio latente) y su negativo más difícil (el recorte de autor distinto más cercano), concentrando la pérdida en los tripletes más informativos del *batch*. Los autores representados por un único recorte se conservan en el entrenamiento, ya que gracias a la fuerte aumentación de los datos, a partir de un solo recorte se generan múltiples ejemplos distintos que pueden actuar como par positivo entre sí.

Las cabezas de estilo se entrenan con la pérdida *Supervised Contrastive* [45] y temperatura $T = 0.07$, optimizador Adam con *learning rate* 10^{-4} y un *scheduler*

cosenoidal a lo largo de 60 épocas. Para que cada *batch* contenga al menos un par positivo por ancla, se utiliza un muestreador balanceado que toma 4 recortes por clase y construye entre 10 y 30 *batches* por época en función del tamaño efectivo del conjunto. El muestreo se realiza con reemplazo, de modo que las clases con menos de 4 recortes repiten ejemplos dentro del *batch* hasta completar la cuota. Se aplica parada temprana con paciencia de 15 épocas sobre la pérdida de entrenamiento, y la cabeza seleccionada para los *benchmarks* es la que minimiza dicha pérdida.

4.5 Reproducibilidad

Más allá del entorno y las versiones de dependencias (sección 3.5), la reproducibilidad se apoya en dos elementos. Primero, todo estimador estocástico se instancia con `random_state` fijo (K-medias, GMM, UMAP y la selección automática de hiperparámetros descrita en 4.3 comparten semilla); además, los modelos de *embeddings* en PyTorch se fuerzan a ser deterministas (`torch.backends.cudnn.deterministic` activado y `benchmark` desactivado) y UMAP se ejecuta con `n_jobs=1` para que el orden de acumulación en paralelo no varíe entre máquinas. Segundo, todos los ficheros de configuración de los experimentos se publican junto con el código, de modo que cualquier resultado descrito puede reproducirse.

5 Resultados y discusión

5.1 Catálogo de experimentos

Antes de discutir los resultados, la tabla 8 resume los experimentos ejecutados. Cada uno varía un único eje alrededor de una configuración base común para que las comparaciones sean justas; el eje variado, los componentes fijos y los tamaños de banco N evaluados se recogen en la tabla. Las subsecciones siguientes se refieren a cada barrido por su identificador.

Id	Elementos variados	Fijo	N / repeticiones
E1	Extractor	UMAP-10, HDBSCAN <code>mcs=5</code> , identidad, SQLite	$N \in \{250, 1000, 6416\}$, $K=3$
E2	Reducción	Extractor base, HDBSCAN <code>mcs=5</code>	$N \in \{250, 1000, 6416\}$, $K=3$
E3	Agrupamiento	Extractor base, UMAP-10	$N \in \{250, 1000, 6416\}$, $K=3$
E4	HDBSCAN <code>min_cluster_size</code>	Extractor base, UMAP-10	$N \in \{250, 1000, 6416\}$, $K=3$
E5	Segmentador	<i>Pipeline</i> base	$N \in \{250, 1000, 6416\}$, $K=3$
E6	Almacenamiento	<i>Pipeline</i> base, búsqueda activa	$N \in \{500, 1000, 2500, 6416\}$, $K=5$
E7	$N \times$ agrupamiento	Extractor base, UMAP-10, SQLite	$N \in \{250, 500, 1000, 2000, 3500, 6416\}$, $K=5$

Id	Elementos variados	Fijo	N / repeticiones
E8a	Extractor $\times$ reducción $\times$ agrupamiento sobre etiquetas de estilo	SQLite, identidad, sin límite	294 recortes, 4 clases, $K=3$
E8b	Extractor $\times$ reducción $\times$ agrupamiento sobre etiquetas de autoría	SQLite, identidad, sin límite	185 recortes, 87 autores, $K=3$
E9	Parámetros HNSW	<i>Pipeline</i> base, PostgreSQL	$N=6416$, $K=5$

Cuadro 8: Catálogo de barridos experimentales. El detalle metodológico común (métricas, cronometría, repeticiones) se describe en 4.

5.2 Calibración del *pipeline*

En esta sección se recorren las decisiones tomadas para el diseño de lo que se considera el *pipeline* base del proyecto, la configuración sobre la que se asientan los estudios de coste y calidad supervisada en las secciones siguientes. Tras esta serie de experimentos, la elección de segmentador, reducción y agrupamiento queda decidida por la triada de experimentos E2, E3 y E4. La excepción a esto es la elección del extractor, que se basa en un criterio de calidad supervisada (E8a) que se discute en la sección de resultados supervisados (5.4).

A lo largo de toda la subsección las métricas intrínsecas, que denominaremos simplemente *calidad intrínseca* (silueta $\uparrow$, CH $\uparrow$, DB $\downarrow$; definidas en 3.2) se leen siempre de forma conjunta con el número de clústeres, la fracción de ruido y el coeficiente de variación de los tamaños de clúster. Tal y como se mencionó en la sección anterior, leer la silueta o el CH de forma aislada lleva a inversiones consistentes del ranking, porque ambos premian configuraciones degeneradas: pocos clústeres grandes y bien separados (CH alto) o particiones binarias triviales (silueta alta). El juicio final se emite siempre sobre el criterio conjunto, no sobre la métrica.

Extractor de características El primer paso del pipeline es la extracción de características. Para esto se prueban doce extractores: cuatro CNN pre-entrenadas en ImageNet (*resnet50*, *vgg16*, *inception_v3*, *mobilenet_v3*), *dinov2_vits14*, *clip_vit_b32*, dos *backbones* YOLO (*yolon*, *yolom*) y la matriz completa 2×2 de cabezas *fine-tuned* (autor y estilo $\times$ DINOv2 y MobileNetV3); la tabla 9 recoge la calidad intrínseca a $N = 6416$.

Extractor	cls	ruido	sil	CH	DB	csCV
mobilenet_v3	820	0.078	0.260	13.1	1.48	0.52
resnet50	794	0.086	0.257	15.6	1.46	0.71
vgg16	773	0.095	0.228	13.4	1.55	0.54
mob._graffiti_author_head	779	0.124	0.227	13.4	1.53	0.53
inception_v3	724	0.118	0.213	11.5	1.61	0.64
dinov2_graffiti_author_head	631	0.143	0.187	17.9	1.59	0.81

Extractor	cls	ruido	sil	CH	DB	csCV
dinov2_vits14	632	0.123	0.185	19.2	1.58	0.92
clip_vit_b32	734	0.150	0.170	11.9	1.66	0.74
mob._graffiti_style_head	629	0.214	0.164	19.7	1.74	0.58
dinov2_graffiti_style_head	562	0.202	0.155	27.9	1.70	0.74
yolom	480	0.253	0.131	32.2	1.75	0.74
yolon	348	0.341	0.085	51.2	1.91	0.81

Cuadro 9: E1, calidad intrínseca a $N = 6416$. Las CNN entrenadas en ImageNet encabezan la silueta; el extractor base (**dinov2_graffiti_style_head**) presenta un CH (27.9) solo superado por los *backbones* YOLO, pese a una silueta inferior a la de su *backbone* DINOv2. Columnas: cls=número de clústeres, ruido=fracción de ruido ↓, sil=silueta ↑, CH=Calinski–Harabász ↑, DB=Davies–Bouldin ↓, csCV=coef. de variación del tamaño de clúster ↓.

Probablemente, el resultado más llamativo del barrido es la inversión entre silueta y Calinski–Harabasz sobre ambas cabezas estilísticas. La silueta cae con respecto a su *backbone* (DINOv2: 0.185 → 0.155; MobileNetV3: 0.260 → 0.164) mientras que el CH sube en los dos casos (DINOv2: 19.2 → 27.9, solo superado por los *backbones* YOLO; MobileNetV3: 13.1 → 19.7). Este fenómeno, aunque contraintuitivo a primera vista, puede explicarse por la naturaleza del entrenamiento contrastivo y los datos utilizados: el *contrastive loss* optimiza la relación entre la compactación de los *clusters* y su separación (CH), pero no la cohesión local de cada punto dentro de su clúster (silueta). En el caso de las cabezas estilísticas, entrenadas con recortes y no con imágenes completas como aquí se evalúan, este cambio de dominio junto con la función de pérdida explicaría la caída de la silueta.

Las CNN entrenadas en ImageNet lideran la silueta (a $N = 6416$: **mobilenet_v3**=0.260, **resnet50**=0.257), con ruido bajo (0.078–0.118) y los clústeres más balanceados (csCV $\approx$ 0.5–0.7). Sin embargo, esto no significa que sean los mejores extractores para el agrupamiento de grafiti: es muy probable que estos modelos estén realizando un buen agrupamiento de apariencia (textura, color, composición) sin que este agrupamiento codifique información relevante para el ámbito del grafiti. YOLO, por el contrario, pierde en todas las categorías a la vez (a excepción del índice CH, que presenta dos de los más altos), confirmando que sus características de detección no transfieren a un agrupamiento basado en apariencia.

Por otro lado, las métricas que describen el coste del barrido figuran en la tabla 10. **mobilenet_v3** es el extractor más barato tanto en velocidad como en memoria gráfica (7.93 ips, 32 MB de VRAM) y la cabeza añade un sobrecoste pequeño ($\sim 8\%$, 7.30 ips). Los *backbones* YOLO son los más caros (4.5–4.9 ips y ~ 825 MB de RSS), y VGG16/CLIP los más exigentes en VRAM (552 y 592 MB).

Extractor	ingesta (s)	ips	RSS pi-co (MB)	VRAM (MB)
mobilenet_v3	808.8	7.93	366.0	31.8
mob._graffiti_author_head	879.1	7.30	406.4	34.5
inception_v3	973.7	6.59	337.2	116.1
vgg16	980.8	6.54	315.0	551.7
resnet50	1080.0	5.94	336.0	117.8
dinov2_vits14	1215.7	5.28	209.6	98.0
dinov2_graffiti_style_head	1245.0	5.15	286.2	99.7
clip_vit_b32	1247.0	5.15	193.1	591.9
yolon	1301.0	4.93	853.8	65.4
yolom	1407.4	4.56	824.9	192.9

Cuadro 10: E1, coste de ingesta a $N = 6416$. La cabeza añade un sobrecoste despreciable sobre el *backbone*; los *backbones* YOLO son los más caros simultáneamente en tiempo y RSS. Se muestran 10 de los 12 extractores (las cabezas `mob._graffiti_style_head` y `dinov2_graffiti_author_head` se omiten por brevedad; su coste cae dentro del rango de la cabeza correspondiente ya presente.)

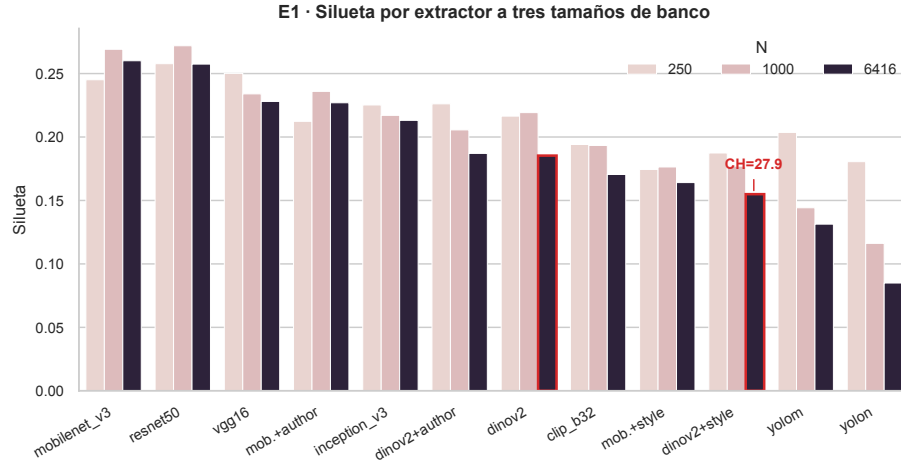


Figura 8. E1: silueta por extractor a tres tamaños de banco. Las CNN entrenadas en ImageNet encabezan la columna, pero la cabeza estilística de DINOv2 invierte silueta/CH (ver texto).

Reducción de dimensionalidad Reducir la dimensionalidad de los vectores de características es un paso común en los *pipelines* de agrupamiento. Sobre los vectores de 128 dimensiones de `dinov2_graffiti_style_head`, el barrido (E2)

compara seis configuraciones y revela a UMAP como la única familia que produce simultáneamente un número de clústeres no trivial, ruido moderado y tamaños balanceados. La tabla 11 resume el resultado al tamaño de dataset completo.

Reducción	cls	ruido	sil	CH	DB	csCV
identidad	301	0.684	0.307	37.5	1.16	0.55
PCA-10	2	0.010	0.000	4.2	1.39	1.00
PCA-50	238	0.698	0.238	36.6	1.29	2.29
UMAP-10	562	0.202	0.155	27.9	1.70	0.74
UMAP-50	558	0.195	0.154	27.8	1.71	0.70
Isomap-10	125	0.624	0.044	38.4	1.80	3.40

Cuadro 11: E2, calidad intrínseca a $N = 6416$. UMAP es la única reducción que produce simultáneamente un número de clústeres no trivial, ruido moderado y tamaños balanceados. Columnas: cls=número de clústeres, ruido=fracción de ruido $\downarrow$, sil=silueta $\uparrow$, CH=Calinski–Harabász $\uparrow$, DB=Davies–Bouldin $\downarrow$, csCV=coef. de variación del tamaño de clúster $\downarrow$.

La fila *identidad* responde directamente a la pregunta de cuánto aporta la reducción de dimensionalidad: agrupando los 128-d crudos, HDBSCAN produce particiones muy distintas según la escala. Colapsa a $k = 2$ a $N = 250$, pero a $N = 1000$ y $N = 6416$ explota a 52 y 301 clústeres respectivamente. Esos clústeres, sin embargo, se obtienen a costa de un ruido altísimo (0.605 y 0.684): HDBSCAN agrupa únicamente un núcleo denso y descarta entre el 60 y el 68% de los puntos. La identidad no produce, por tanto, un colapso trivial, pero tampoco una partición utilizable bajo el criterio conjunto, sino una segmentación de cobertura muy baja. UMAP-10 sobre los mismos datos escala a 18, 106 y 562 clústeres manteniendo el ruido moderado (0.06, 0.11, 0.20) y los tamaños balanceados. PCA-10 y PCA-50 ilustran otra inestabilidad: PCA-10 colapsa a $k = 2$ con silueta nula (0.000) a $N = 6416$ y desciende a silueta negativa (-0.020) a $N = 1000$, mientras que PCA-50 explota a $k = 238$ con un 69.8% de ruido y un coeficiente de variación de 2.29. Ambas son configuraciones degeneradas que las métricas individuales premian engañosamente: tanto la identidad (silueta 0.307, DB 1.16) como PCA-50 (silueta 0.238) encabezan la tabla en silueta pese a descartar dos tercios del dataset.

Las métricas de coste refuerzan la posición de la identidad como opción inferior. Esta reducción no es gratuita como podría parecer, sino que empuja el coste hacia la etapa de agrupamiento. Con $N = 6416$ imágenes, HDBSCAN tarda 6.83 s sobre la representación identidad frente a 0.34 s sobre UMAP-10, unas 20 veces más. Isomap-10 fracasa en la misma vía, pero de forma más grave: 23.9 s y 1.25 GB de RSS pico, lo más caro del experimento, para una partición igualmente degenerada (ruido 0.624 y csCV 3.40 a $N = 6416$, con colapso a $k = 2$ a $N = 1000$). UMAP-10 y UMAP-50 son indistinguibles en calidad, pero UMAP-50 cuesta casi el doble en tiempo de reducción (11.0 s vs 5.9 s a $N = 6416$). Por esto, la recomendación es específicamente UMAP con `n_components=10`, no UMAP en general.

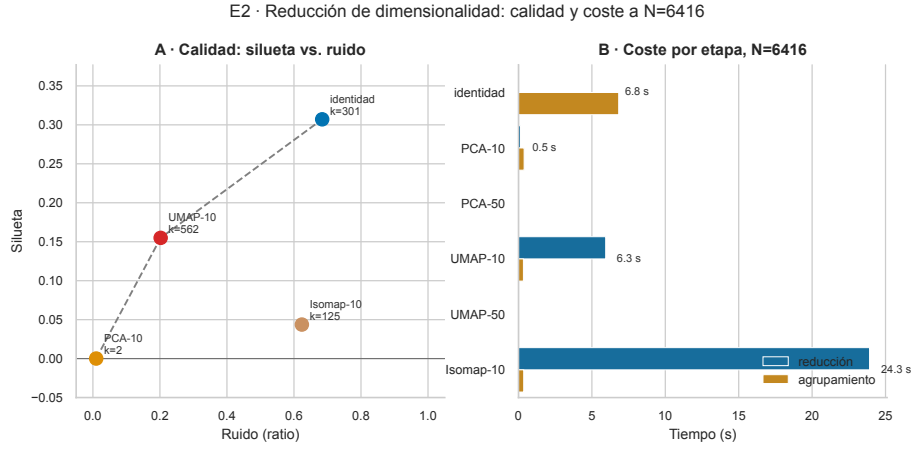


Figura 9. E2: calidad intrínseca (panel izquierdo) y coste por etapa (panel derecho) por familia de reducción a $N = 6416$. La fila identidad produce una partición de cobertura muy baja (ruido ~ 0.68) y traslada el coste a la etapa de *clustering*.

Algoritmo de agrupamiento HDBSCAN es el único agrupador del panel que combina una silueta competitiva, escalado sensato del número de clústeres con N , ruido moderado, tamaños balanceados y coste despreciable bajo el criterio conjunto. El barrido (E3) cubre 14 configuraciones sobre el extractor base y UMAP-10: HDBSCAN, K-medias (ajustado con el método del codo), GMM diagonal (BIC con tres techos), DBSCAN (auto- ε y tres ε fijos), OPTICS coseno, aglomerativo (*average linkage*, $k \in \{10, 20\}$) y espectral ($k \in \{10, 20\}$); la tabla 12 extrae la fila representativa de cada familia.

Algoritmo	cls	ruido	sil	CH	DB	csCV	clu (s)	mem (MB)
HDBSCAN (mcs=5)	563	0.199	0.151	33	1.70	0.75	0.348	0.3
OPTICS (cos)	648	0.247	0.177	30	1.64	0.45	8.578	763.3
GMM · 300 (BIC)	297	0.000	0.054	50	2.52	0.58	1.839	49.7
K-medias (codo, $k = 8$)	8	0.000	0.057	805	3.23	0.37	0.009	0.2
DBSCAN auto- ε	239	0.059	-0.015	46	1.97	2.47	0.080	0.2
Aglom. <i>avg</i> $k = 10$	10	0.000	-0.015	506	3.11	1.36	0.408	314.2
Espectral $k = 10$	10	0.000	-0.117	264	3.33	1.56	2.073	1.1

Algoritmo	cls	ruido	sil	CH	DB	csCV	clu (s)	mem (MB)
-----------	-----	-------	-----	----	----	------	------------	-------------

Cuadro 12: E3, fila representativa de cada familia a $N = 6416$. La columna **mem** reporta el pico de RSS delta de la etapa de agrupamiento. Sólo HDBSCAN combina un número no trivial de clústeres, ruido moderado, tamaños balanceados, coste despreciable y huella de memoria mínima. Columnas: cls=número de clústeres, ruido=fracción de ruido ↓, sil=silueta ↑, CH=Calinski–Harabász ↑, DB=Davies–Bouldin ↓, csCV=coef. de variación del tamaño de clúster ↓. *clu*=tiempo de agrupamiento, *mem*=pico de RSS delta.

Como se anticipó en 5.2, es necesario leer las métricas intrínsecas en conjunto con el número de clústeres, el ruido y el csCV; nunca de manera aislada. Esto es especialmente importante aquí puesto que los algoritmos basados en densidad (HDBSCAN, DBSCAN, OPTICS) son juzgados solo sobre el subconjunto que agrupan, mientras que los particionales (K-medias, GMM, aglomerativo, espectral) lo son sobre el conjunto de datos completo. La silueta penaliza menos a un método que se abstiene de los puntos difíciles y CH premia la compactación, sin importar la estructura.

Bajo este criterio, HDBSCAN parece ser el único algoritmo que produce resultados defendibles en el rango de escalas evaluado: obtiene un ruido moderado y un balance de tamaños razonable, una silueta competitiva y un coste despreciable. OPTICS es su único par en calidad (superándolo ligeramente en todas las escalas en silueta y sustancialmente en variación de tamaños), pero se ve completamente dominado en coste, con un tiempo de ejecución ~ 25 veces mayor y un uso de memoria ~ 2500 veces mayor que HDBSCAN a $N = 6416$.

Por otro lado, entre los métodos particionales, GMM con selección de k por BIC es la mejor alternativa si es necesaria una agrupación sin ruido. El resto de métodos quedan dominados por baja calidad (DBSCAN es inestable, espectral presenta la peor silueta del panel) o por el escalado de memoria.

Granularidad de HDBSCAN Una vez escogido HDBSCAN como algoritmo de agrupamiento, el siguiente paso es calibrar su hiperparámetro de granularidad `min_cluster_size` (*mcs*). Este parámetro controla el número mínimo de puntos que debe tener un clúster para ser considerado como tal, y por tanto afecta directamente al número de clústeres, la fracción de ruido y el balance de tamaños. El barrido (E4) cubre seis valores de *mcs* entre 5 y 200, como se puede ver en la tabla 13 a $N = 6416$.

<i>mcs</i>	cls	ruido	sil	CH	DB	csCV
5	563	0.199	0.151	33	1.70	0.75
10	154	0.277	0.037	56	2.20	1.78
25	26	0.228	0.047	209	2.66	1.81
50	4	0.026	0.193	1404	2.01	0.86
100	2	0.008	0.269	2765	1.27	0.40

mcs	cls	ruido	sil	CH	DB	csCV
200	2	0.011	0.270	2776	1.27	0.40

Cuadro 13: E4, barrido de `min_cluster_size` a $N = 6416$. La silueta y el CH alcanzan su máximo en los valores degenerados ($k=2$); el criterio conjunto sitúa el óptimo en `mcs=5`. Columnas: `cls`=número de clústeres, `ruido`=fracción de ruido ↓, `sil`=silueta ↑, `CH`=Calinski–Harabász ↑, `DB`=Davies–Bouldin ↓, `csCV`=coef. de variación del tamaño de clúster ↓.

Los valores más altos de `mcs` (100 y 200), aunque presentan las métricas intrínsecas más altas con tasas de ruido y variación de tamaños bajas, son claramente degenerados, puesto que solo producen dos clústeres masivos que absorben casi todos los puntos. El caso de 50 es, aunque muy similar, algo más defendible. El modelo utilizado para la generación de los *embeddings* (`dinov2_graffiti_style_head`) fue entrenado con cuatro categorías de estilo, por lo que esta segmentación a cuatro clústeres podría ser interpretada como un resultado razonable.

Los siguientes dos valores, 25 y 10 respectivamente, presentan un número de clústeres mayor, más segmentado, pero a costa de una caída considerable en todas las métricas intrínsecas y un aumento de ruido y variación de tamaños. El caso de `mcs=5`, presenta un número de clústeres aún mayor, pero con menor ruido y métricas superiores a los dos casos anteriores (aunque no mejores que los valores degenerados). Se escoge `mcs=5` como valor base para el resto del proyecto por una preferencia por un número de clústeres más alto y la sospecha de que el caso para `mcs=50` es un caso de agrupamiento demasiado grueso. Esta decisión se ve reforzada en el experimento E8a, donde las métricas supervisadas favorecen a la configuración escogida frente a la otra.

Impacto del segmentador En principio, el recorte de la imagen completa en unidades más pequeñas que contengan únicamente un grafiti debería facilitar el agrupamiento, al eliminar ruido de fondo y permitir que el extractor de características se centre en la información relevante. Este cambio debería ser especialmente beneficioso para los extractores entrenados con recortes (las cabezas de DINOv2 y MobileNetV3). El barrido E5 compara cuatro opciones de segmentación (imagen completa, YOLO11s, YOLO11m y YOLO11m *fine-tuned*) manteniendo fijo el resto del *pipeline*.

Segmentador	recortes	r/img	cls	ruido	sil	CH	ingesta (s)
identidad	6416	1.00	563	0.199	0.151	33.1	1235.5
YOLO11s	6590	1.03	502	0.225	0.127	30.4	1777.0
YOLO11m	6603	1.03	506	0.230	0.131	29.5	1811.6
YOLO11m <i>ft</i>	15362	2.39	1049	0.298	0.122	36.2	2152.3

Segmentador	recortes	r/img	cls	ruido	sil	CH	ingesta (s)
-------------	----------	-------	-----	-------	-----	----	-------------

Cuadro 14: E5, calidad intrínseca y coste de ingesta a $N = 6416$ (la columna «recortes» son los puntos efectivamente agrupados, no las imágenes leídas). Cada segmentador alimenta a HDBSCAN un conjunto distinto de puntos. Columnas: r/img=recortes por imagen, cls=número de clústeres, ruido=fracción de ruido ↓, sil=silueta ↑, CH=Calinski-Harabász ↑, ingesta=tiempo de ingesta.

A escala completa, la identidad gana en todos los ejes que importan: silueta más alta (0.151), ruido más bajo (0.199) y mejor coste de ingesta (1235 s vs 1777–2152 s). Los detectores base (YOLO11s y YOLO11m) recortan ~ 1.03 cajas por imagen, prácticamente reproduciendo el conjunto original con una ganancia marginal o negativa en calidad y un sobrecoste de ingesta del 44%.

El detector *fine-tuned* es un caso interesante: a $N = 1000$ obtiene la mejor silueta, mejor CH y mejor DB del panel, pero este liderazgo desaparece con $N = 6416$, donde obtiene casi 2.4 recortes sobre cada imagen original y degenera el agrupamiento, con un 29.8% de ruido, una silueta de 0.122 y un coeficiente de variación de tamaños de 1.06, con 1049 clústeres y una cola larga de clústeres pequeños. El coste de ingesta también se dispara a 2152 s, un 74% más que la identidad.

Las razones de esta caída de calidad se vuelven más claras al consultar las métricas de validación del modelo de detección. En primer lugar, este obtiene una precisión de 0.849 y un recall de 0.741. Esto implica que, cuando el modelo detecta un grafiti, lo hace correctamente el 84.9% de las veces, pero también que no anota uno de cada cuatro grafitis presentes en la imagen. Por otro lado, obtiene valores de 0.808 y 0.603 para mAP@0.5 y mAP@0.5-0.95 respectivamente. Esto nos indica que, incluso en los casos en los que el modelo detecta un grafiti, la caja delimitadora no se ajusta con mucha precisión al valor real, lo que sugiere que los recortes pueden ser de calidad variable.

Con estos datos en mente, es posible que el modelo *fine-tuned* produzca recortes de calidad muy variable, muchas veces llegando a no recortar la imagen y devolviéndola completa o dejando atrás varios grafitis. Esta variabilidad en los datos de entrada, junto con el hecho de que el extractor base no fue entrenado con recortes de este tipo, podría ser la causa de la caída de calidad y el aumento de coste a medida que el tamaño del banco crece. En la figura 12 se muestran ejemplos de detecciones del modelo obtenidas durante el proceso de entrenamiento.

Esta conclusión cierra la configuración base del trabajo y la fija como: `dinov2_graffiti_style_head + UMAP-10 + HDBSCAN mcs=5 + segmentador identidad + SQLite`. Es la configuración sobre la que se asientan las subsecciones siguientes.

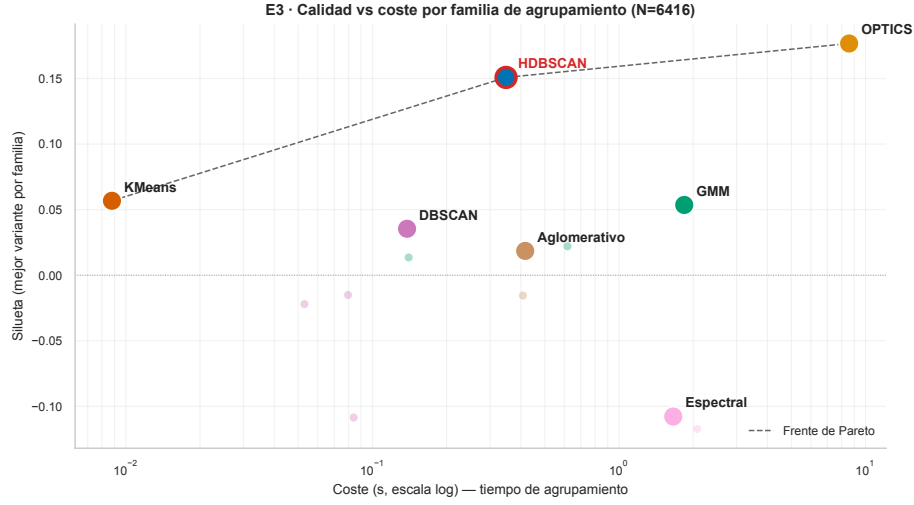


Figura 10. E3: calidad frente a coste por familia a $N = 6416$ (eje horizontal logarítmico). Cada familia se representa por su mejor variante en silueta (marcador grande); los puntos pequeños son el resto de variantes. La línea discontinua marca el frente de Pareto: HDBSCAN (rojo) domina a todas excepto a OPTICS, que paga ~ 25 veces más coste por ~ 0.02 más de silueta, y K-medias, el algoritmo más barato pero con una silueta 0.094 puntos inferior a la de HDBSCAN.

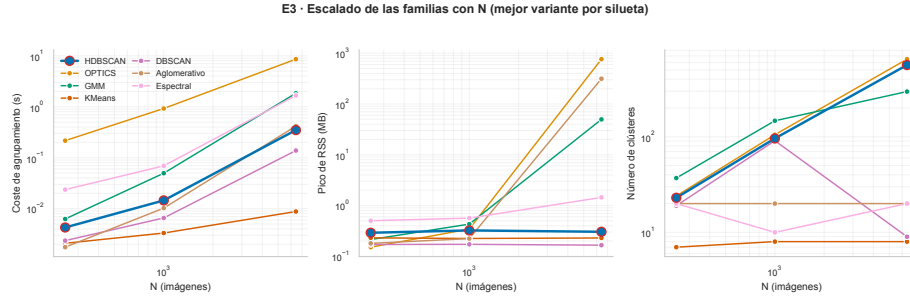


Figura 11. E3: escalado con el tamaño del banco N por familia. Para cada N se selecciona la variante de mejor silueta dentro de la familia. El coste (izquierda) y la huella de memoria (centro) crecen varios órdenes de magnitud para OPTICS y aglomerativo mientras HDBSCAN se mantiene plano. El número de clústeres (derecha) crece de forma sensata para HDBSCAN, OPTICS y GMM, frente a los métodos con k fijo (K-medias, aglomerativo, espectral) o inestables (DBSCAN).

5.3 Coste computacional

Este es uno de los ejes centrales del trabajo: caracterizar empíricamente cómo crece el coste de cada etapa con el tamaño del banco y contrastar las pendientes observadas con las clases de complejidad teóricas. El rango disponible ($N \leq$



Figura 12. Ejemplos de detecciones del detector de YOLO durante el entrenamiento. Este es uno de los «mejores» casos, en los que la gran mayoría de los grafitis son identificados.

6416) es estrecho para separar asintóticamente $O(n \log n)$ de $O(n^2)$, por lo que las pendientes se reportan como *observadas en este régimen* y no como exponentes asintóticos.

Escalabilidad por etapa El barrido de escalabilidad (E7) mide ingesta, reducción, agrupamiento (cinco algoritmos) y búsqueda por similitud sobre seis tamaños de banco; la tabla 15 consolida los tiempos por etapa a las tres escalas extremas con HDBSCAN como agrupador de referencia, y la tabla 16 contrasta las pendientes log-log observadas con la complejidad teórica de referencia.

N	ingesta (s)	red (s)	HDBSCAN (s)	K-medias (s)	aglo (s)	búsqueda (s)
250	58.7	0.253	0.0038	0.0025	0.0019	3.66
500	115.7	0.603	0.0069	0.0027	0.0046	16.32
1000	231.6	1.514	0.0144	0.0033	0.0117	63.28
2000	455.9	4.542	0.1106	0.0047	0.0427	306.81
3500	776.0	11.870	0.1863	0.0066	0.1405	995.52
6416	1294.9	6.237	0.3580	0.0098	0.5988	3266.60

Cuadro 15: E7, tiempo por etapa frente a N (extracto). A $N = 6416$ la búsqueda por similitud SQLite domina absolutamente, con la ingesta como segundo coste. La caída de UMAP a $N = 6416$ se discute en el texto.

Etapas / Algoritmo	Escala teórica	Pendiente $N=250 \dots 6416$	Pendiente $N=500 \dots 3500$
Ingesta (DINOv2)	$O(n)$	0.95	–
Reducción (UMAP)	$\sim O(n^{1.14})$	1.15	1.53
K-medias (k fijo)	$\approx O(n)$	0.43	0.46
DBSCAN	$O(n \log n) - O(n^2)$	0.99	1.10
OPTICS	$O(n \log n) - O(n^2)$	0.94	0.98
HDBSCAN	$O(n \log n)$ amort.	1.53	1.84
Aglomerativo	$O(n^2 \log n)$	1.77	1.76
Búsqueda (SQLite)	$O(n^2)$ por pasada	2.10	2.13

Cuadro 16: E7, pendientes observadas (regresión log-log) por etapa, comparadas con la complejidad teórica de referencia. La ventana 500...3500 excluye el punto $N = 6416$ de reducción (anomalía `pynndescent`, ver texto).

Como se puede ver en la segunda tabla, el rango reducido de N provoca que varias de las etapas tengan pendientes observadas menores que la complejidad teórica de referencia, como puede ser el caso de DBSCAN o OPTICS. Algoritmos como HDBSCAN o el aglomerativo, con complejidades más altas, sí que presentan pendientes superiores a 1, dejando clara su naturaleza super-lineal. El tiempo de ejecución de estos algoritmos fue prácticamente despreciable al compararlo con la ingesta o la búsqueda por similitud, no llegando a superar los 5 s en ningún caso. El único cuello de botella para el agrupamiento a esta escala es el aglomerativo, que a $N = 6416$ consume 314.1 MB de RAM, creciendo como $O(n^2)$ y descartándolo operativamente mucho antes de que su tiempo de pared lo haga.

La pendiente observada de la búsqueda por similitud SQLite es prácticamente la esperada: ≈ 2.10 sobre el rango completo y 2.13 en la ventana limpia. A $N = 6416$ una sola pasada de 6416 consultas sobre el dataset tarda 54.4 minutos, más que la propia ingesta (21.6 min) y unas 9000 veces más que el agrupamiento (3266.6/0.358 s). El *throughput* cae de 68 consultas por segundo a $N = 250$ a 2 consultas por segundo a $N = 6416$: una caída de 35 veces. Este resultado es la motivación cuantitativa directa para el barrido de *backend* y de índice ANN (5.3).

La anomalía más importante del barrido se encuentra en la ingesta para el algoritmo de reducción UMAP. Este, crece hasta $N = 3500$ con una pendiente de 1.15, pero a $N = 6416$ cae a 6.24 s, un 47% más rápido sobre un 83% más de datos. Esta caída es reproducible y se debe a que UMAP utiliza un umbral interno para cambiar a un cálculo aproximado de vecinos basado en `pynndescent` al superar unos pocos miles de puntos. Esto provoca que la pendiente completa no describa un único régimen, sino dos, y es la razón por la que se reportan dos pendientes distintas en la tabla 16.

E7 · Escalabilidad temporal (log-log)

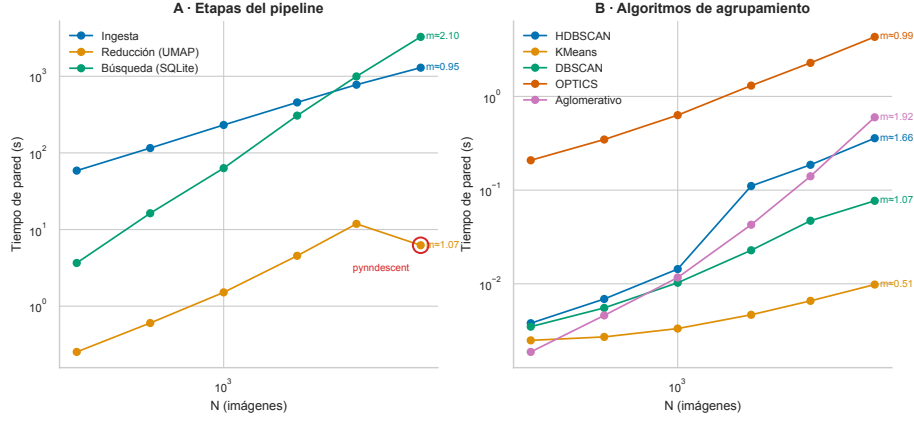


Figura 13. E7 (figura titular): tiempo de pared vs N por etapa del *pipeline*, escala log-log. La pendiente de la búsqueda por similitud SQLite (≈ 2.10) confirma el coste $O(n^2)$ por pasada. La caída de UMAP a $N = 6416$ refleja el cambio interno a *pyndescent*.

E7 · Memoria pico por etapa

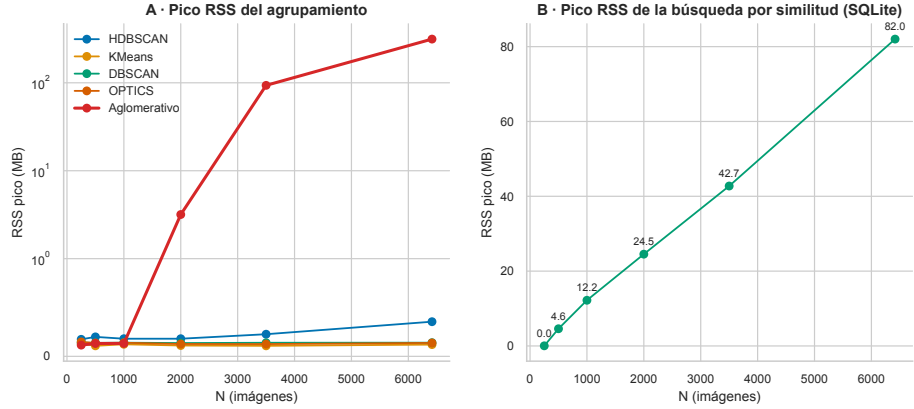


Figura 14. E7: memoria pico por etapa. El aglomerativo materializa la matriz $O(n^2)$ de distancias y crece cuadráticamente con N ; HDBSCAN, DBSCAN y OPTICS se mantienen sublineales.

Búsqueda por similitud: *backend* e índice ANN El cuello $O(n^2)$ de la búsqueda SQLite identificado en E7 motiva el barrido E6 sobre tres *backends*: SQLite (exploración lineal en Python), pgvector exacto (escaneo secuencial dentro del SGBD con el *kernel* de coseno en C) y pgvector con índice HNSW ($m=16$, $ef_construction=64$). La construcción del índice HNSW se cronometra una so-

la vez entre la ingesta y el bucle de búsqueda, así que su coste no se acumula sobre el tiempo de consulta.

N	Backend	ss (s)	ips	idx (s)	R@5
500	SQLite	3.05	164	–	–
500	PostgreSQL exacto	0.162	3 096	0.0	–
500	PostgreSQL + HNSW	0.164	3 042	0.066	1.000
1000	SQLite	6.41	156	–	–
1000	PostgreSQL exacto	0.190	5 257	0.0	–
1000	PostgreSQL + HNSW	0.187	5 353	0.136	1.000
2500	SQLite	16.89	148	–	–
2500	PostgreSQL exacto	0.234	10 678	0.0	–
2500	PostgreSQL + HNSW	0.166	15 092	0.372	1.000
6416	SQLite	43.10	149	–	–
6416	PostgreSQL exacto	0.578	11 109	0.0	–
6416	PostgreSQL + HNSW	0.179	35 936	0.465	1.000

Cuadro 17: E6, latencia, *throughput* y *recall* de la búsqueda por similitud por *backend* y tamaño de banco. SQLite escala linealmente en tiempo por consulta; pgvector exacto está dominado por la latencia de *round-trip* en los tamaños pequeños; HNSW solo se separa visiblemente por encima de $N \approx 2500$.

SQLite mantiene un *throughput* aproximadamente plano (~ 150 ips) con tiempo de ejecución lineal en N . PostgreSQL exacto eleva el *throughput* en dos órdenes de magnitud. Por debajo de $N = 1000$ la latencia está dominada por el *round-trip* de la conexión. PostgreSQL con HNSW es el único *backend* cuyo *throughput* crece con el tamaño del banco con respecto al exacto: a $N = 2500$ es 100 veces más rápido que SQLite y a $N = 6416$ es 3.2 veces más rápido que el exacto y 240 veces más rápido que SQLite, con R@5 = 1.000 a las cuatro escalas. La construcción del índice cuesta 0.47 s a $N = 6416$, se amortiza en una sola consulta frente a SQLite y en varias docenas frente al exacto.

Por debajo de $N \approx 2500$ las tres opciones sobre PostgreSQL son indistinguibles en latencia y HNSW solo añade complejidad sin ganancia; por encima, HNSW pasa al frente mientras el exacto sigue creciendo linealmente. El dataset del trabajo está por encima de ese cruce, por lo que HNSW es la elección por defecto recomendada para la búsqueda; SQLite queda como *backend* de desarrollo de muy pequeña escala.

Para validar que la configuración por defecto del índice no es una decisión arbitraria, un barrido complementario E9 recorre once configuraciones en torno

a la utilizada en el barrido E6 ($m = 16$, $ef_construction = 64$, $ef_search = 40$) sobre el conjunto de datos completo. Los resultados se pueden ver en la figura 15. Ninguna cae por debajo de *recall* 0.990, y el ancla proporciona 0.321 s por consulta a *recall* 0.995, 3.4 veces más rápida que el método exacto. Alcanzar *recall* 1.000 sin penalizar la latencia es posible subiendo m a 32 o $ef_construction$ a 256; ambos son parámetros para la construcción, preferibles a ef_search en un flujo de «ingesta única, muchas consultas» porque su sobrecoste se amortiza una sola vez. A esta escala, parece que el uso de HNSW proporciona una mejora de latencia muy considerable sin una pérdida apreciable de calidad, eliminando el cuello de botella $O(n^2)$ de SQLite.

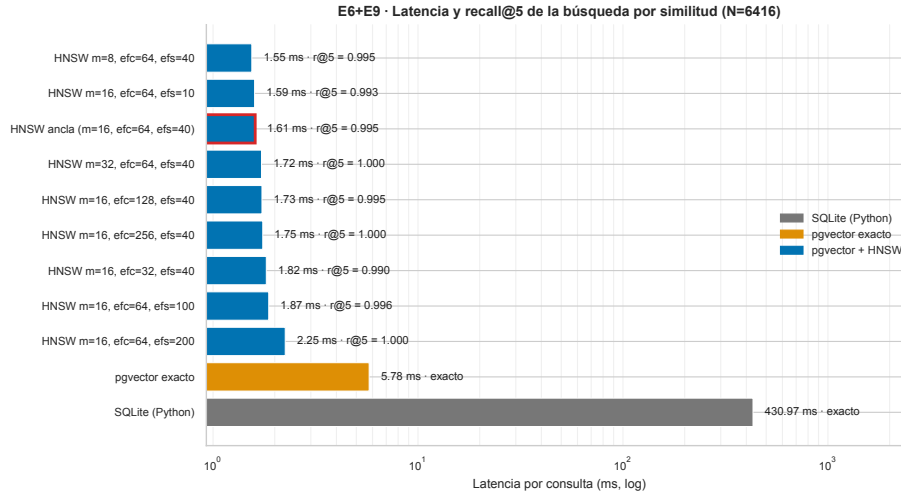


Figura 15. E6+E9: latencia por consulta (eje logarítmico) y *recall*@5 por *backend* y configuración HNSW a $N = 6416$, una fila por configuración ordenada por latencia (cada barra anota su latencia y, para HNSW, el *recall*). La configuración del barrido E6 aparece en rojo; SQLite y pgvector exacto se incluyen como referencia.

5.4 Validación supervisada

Las métricas extrínsecas, aplicables cuando se dispone de etiquetas sobre los pares, las métricas extrínsecas miden directamente la concordancia entre los clústeres y la verdad de referencia. Esta subsección hace uso de los dos conjuntos etiquetados manualmente por autor y estilo para evaluar la capacidad de los extractores de características para separar ambos ejes y validar las métricas intrínsecas empleadas en el resto del trabajo. La unidad de análisis es el recorte (recortes pre-segmentados, segmentador identidad): 294 recortes sobre 4 estilos para E8a y 185 recortes sobre 87 autores para E8b. Los dos diseños no son

simétricos en su exposición al desplazamiento de dominio. La cabeza estilística se entrenó sobre un conjunto *mixto* de 385 recortes (110 Salamanca + 275 Cuenca) y se evalúa sobre 294 recortes de Salamanca disjuntos del entrenamiento, así que E8a es un *held-out* dentro del propio dominio de entrenamiento. Las cabezas de autoría, en cambio, se entrenaron sobre 321 recortes *exclusivamente* de Cuenca y se evalúan sobre 185 recortes de Salamanca, así que E8b mezcla generalización a autores no vistos con un desplazamiento de dominio Cuenca→Salamanca.

Evaluación por estilo El barrido E8a evalúa la capacidad de los distintos extractores para separar los cuatro estilos de grafiti escogidos. La tabla 18 recoge la mejor celda por extractor (E8a) sobre la malla de 2 reducciones $\times$ 6 agrupadores, seleccionada por el ARI por defecto (sin descartar puntos de ruido), *chance-corrected* y, por tanto, comparable entre configuraciones con distinto número de clústeres.

Extractor	ARI	NMI	F1	sil	Celda
dinov2_graffiti_style_head	0.722	0.678	0.806	0.284	identidad + K-medias-4
mob._graffiti_style_head	0.627	0.539	0.740	0.272	identidad + K-medias-4
clip_vit_b32	0.368	0.392	0.556	0.047	UMAP + aglomerativo
dinov2_vits14	0.356	0.374	0.558	0.135	identidad + K-medias-4
mobilenet_v3	0.356	0.336	0.550	0.038	UMAP + aglomerativo
resnet50	0.334	0.277	0.548	0.058	identidad + espectral
mob._graffiti_author_head	0.318	0.295	0.517	0.046	UMAP + aglomerativo
dinov2_graffiti_author_head	0.302	0.273	0.519	0.094	UMAP + aglomerativo

Cuadro 18: E8a, mejor celda por extractor sobre la evaluación de estilo (294 recortes, 4 clases). El ranking sitúa las cabezas de autoría por debajo del *backbone* en ambas familias, aunque con un margen modesto (−0.054 DINOv2, −0.038 MobileNetV3).

El resultado es inequívoco: la cabeza estilística de DINOv2 + identidad + K-medias-4 obtiene $ARI = 0.722$ y $F1 = 0.806$, presenta más del doble del ARI del *backbone* y de cualquier otro extractor sin entrenar. Va seguido por la cabeza estilística de MobileNetV3, que, aunque no llega a doblar el ARI del siguiente extractor, sí lo mejora sustancialmente (0.627 vs 0.368). El resto de extractores sin entrenar (CLIP, DINOv2 y MobileNetV3) se agrupan en un segundo bloque con ARI entre 0.33 y 0.37.

Las cabezas de autoría no solo no mejoran el ARI de estilo respecto a su propio *backbone* sino que lo empeoran (DINOv2 $0.356 \rightarrow 0.302$, MobileNetV3 $0.356 \rightarrow 0.318$), aunque con un margen modesto que las sitúa en los puestos finales del panel. El *fine-tuning* no produce una representación genérica para el grafiti: concentra el eje para el que fue entrenada y deja al otro en clara desventaja. Esta matriz de transferencia diagonal-dominante es la evidencia más limpia de que las dos cabezas codifican información genuinamente distinta.

La señal secundaria de recuperación supervisada (búsqueda por similitud con $k = 5$, evaluada frente a la etiqueta de la consulta) confirma el ordenamiento

y prescinde de la elección de agrupador. La tabla 19 muestra que la cabeza estilística de DINOv2 ocupa la primera posición en los tres índices ($P@5 = 0.851$, $MAP@5 = 0.909$, $MRR = 0.922$) y que el orden por $P@5$ reproduce, en grandes trazos, el ranking por ARI: cabezas de estilo arriba, *backbones* sin entrenar en el bloque intermedio y cabezas de autoría en la parte baja. La cabeza de autor de MobileNetV3 es el peor recuperador del panel (0.639), consistente con su caída de ARI por debajo del propio *backbone*.

Extractor	P@5	MAP@5	MRR
dinov2_graffiti_style_head	0.851	0.909	0.922
mob._graffiti_style_head	0.760	0.818	0.835
dinov2_vits14	0.699	0.805	0.837
mobilenet_v3	0.687	0.806	0.830
dinov2_graffiti_author_head	0.674	0.804	0.832
clip_vit_b32	0.669	0.792	0.825
resnet50	0.654	0.789	0.829
mob._graffiti_author_head	0.639	0.775	0.799

Cuadro 19: E8a, recuperación supervisada *top-5* por extractor sobre la evaluación de estilo. Reproduce el mismo ordenamiento que el ARI de la tabla 18, lo que descarta que el liderazgo de la cabeza estilística esté inducido por la elección de agrupador.

HDBSCAN no parece llegar a distinguir las cuatro categorías de estilo posibles como clústeres separados, sino que las agrupa en tres grupos. Tras una revisión manual de los clústeres generados, parece que HDBSCAN agrupa los grafitis en «sencillos», «elaborados» y «characters» (color naranja, verde y azul en la figura 16). El primero de estos grupos, con unos 150 recortes, está principalmente formado por *tags* y algunos *throw-ups* más simples (figura 17). El segundo, con aproximadamente 120 recortes, está formado por *throw-ups* más elaborados y la gran mayoría de los *pieces* (figura 18). El tercero y el más pequeño con diferencia, con solo 17 recortes, está formado por los *characters* de estilo cómic, manga o grafiti de personajes (figura 19).

Evaluación por autor El barrido E8b evalúa la capacidad de los extractores para separar un conjunto de imágenes por autoría. Se parte de una hipótesis simétrica a la de E8a: una cabeza entrenada para autoría podría superar a su propio *backbone* por bastante margen, y superar a los extractores sin entrenar. La tabla 20 muestra que esta hipótesis no es correcta. El mejor ARI global lo obtiene CLIP ViT-B/32 (0.076), y los ocho extractores caben en una banda 0.041–0.076 cercana al agrupamiento aleatorio (la celda ganadora de E8a vale 0.722, ~ 10 veces más). Dentro de la familia DINOv2 la cabeza de autoría sí supera a su *backbone* (0.065 vs. 0.055) y a la cabeza de estilo (0.054), aunque a escala dominada por ruido; en la familia MobileNetV3 la cabeza apenas iguala a su *backbone* (0.048 vs. 0.046, +0.002 ARI) y supera a la cabeza de estilo (0.041).

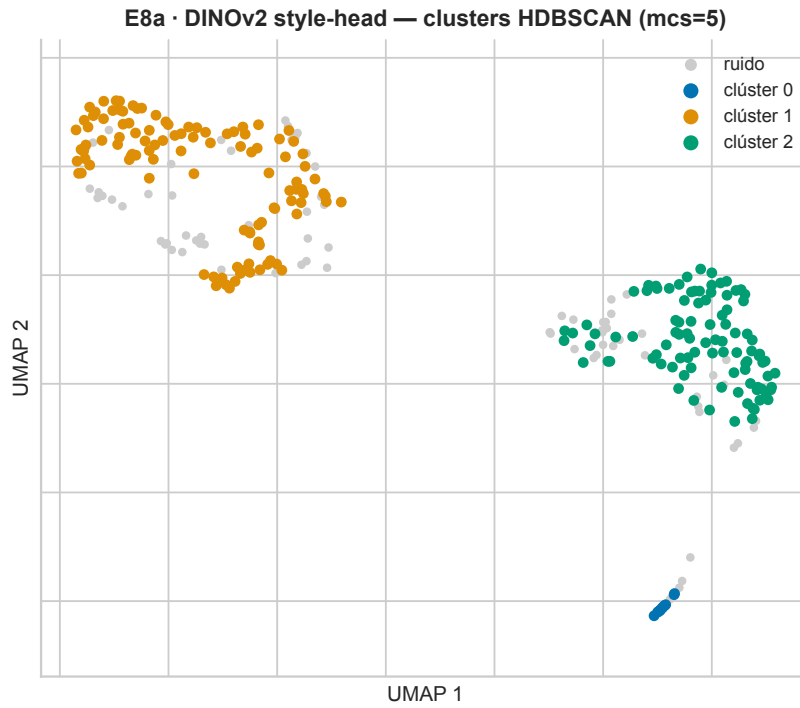


Figura 16. Representación bidimensional de la distribución de los recortes de E8a en el espacio de características de la cabeza estilística de DINOv2, coloreados por los tres clústeres generados por HDBSCAN.

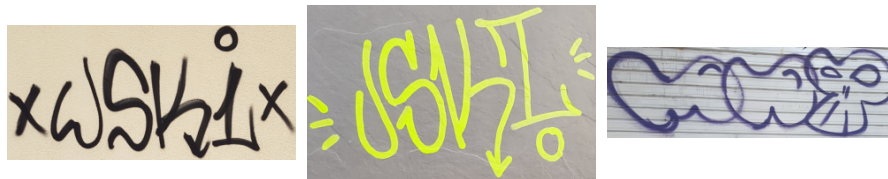


Figura 17. Ejemplo del clúster de sencillo.

Extractor	ARI	NMI	F1	sil	Celda
clip_vit_b32	0.076	0.789	0.090	0.003	UMAP + aglo-87
dinov2_graffiti_author_head	0.065	0.647	0.093	0.052	UMAP + aglo-30
dinov2_vits14	0.055	0.754	0.079	0.096	identidad + aglo-87
dinov2_graffiti_style_head	0.054	0.628	0.085	0.037	UMAP + aglo-30
resnet50	0.049	0.778	0.064	0.028	UMAP + aglo-87
mob._graffiti_author_head	0.048	0.627	0.077	0.021	UMAP + aglo-30
mobilenet_v3	0.046	0.780	0.060	0.023	UMAP + aglo-87

Extractor	ARI	NMI	F1	sil	Celda
mob._graffiti_style_head	0.041	0.629	0.070	0.020	UMAP + aglo-30

Cuadro 20: E8b, mejor celda por extractor sobre la evaluación de autoría (185 recortes, 87 autores, evaluación entre ciudades Cuenca→Salamanca). Todos los ARI caen en una banda estrecha cerca al suelo; la simetría esperada con E8a no se materializa.

E8b es un resultado negativo, pero esperado. La tarea de clasificación por estilo es considerablemente más sencilla que la de agrupamiento por autoría, que además se ve afectada por un desplazamiento de dominio más severo y un conjunto de datos de peor calidad. Sería necesario anotar muchos más recortes y de más autores (probablemente más de los que hay en el dataset completo) para obtener resultados de agrupamiento por autoría aceptables. Otra posibilidad sería pre-entrenar los modelos de autoría sobre un conjunto de datos de grafiti de manera no supervisada, y luego hacer el *fine-tuning* supervisado sobre el conjunto de autoría anotado. Esto último requeriría una gran cantidad de imágenes de grafiti y el uso del dataset completo podría ser un buen punto de partida, pero su uso provocaría un sesgo de selección importante al evaluar sobre el mismo conjunto utilizado para pre-entrenar la red.

El problema de recuperación sigue la misma tendencia que el anterior, empeorando aún más la posición de las cabezas de autoría. CLIP sigue siendo el mejor recuperador ($P@5 = 0.091$), seguido de cerca por ResNet50 (0.085), MobileNetV3 (0.083), DINOv2 base (0.081) y las dos cabezas de autoría (MobileNetV3 0.078, DINOv2 0.077): todos dentro de un rango estrecho. En este caso, las cabezas entrenadas para esta tarea específica se sitúan por debajo de los extractores sin entrenar, lo que es consistente con la caída de ARI y con la hipótesis de que el *fine-tuning* para autoría no consigue una representación útil para esta tarea.

Extractor	P@5	MAP@5	MRR	R@5
clip_vit_b32	0.091	0.252	0.262	0.268
resnet50	0.085	0.246	0.256	0.236
mobilenet_v3	0.083	0.240	0.245	0.251
dinov2_vits14	0.081	0.248	0.251	0.268
mob._graffiti_author_head	0.078	0.217	0.222	0.233
dinov2_graffiti_author_head	0.077	0.228	0.232	0.254
mob._graffiti_style_head	0.071	0.196	0.200	0.176
dinov2_graffiti_style_head	0.065	0.226	0.229	0.192

Cuadro 21: E8b, recuperación supervisada *top-5* por extractor sobre la evaluación de autoría. Línea base aleatoria $P@5 \approx 0.024$. El campo cabe en una banda estrecha; las cabezas de autoría quedan en la zona media en ambos ejes (ARI y $P@5$), coherentemente con un soporte parcial a piso aleatorio.



Figura 18. Ejemplo del clúster de elaborado.



Figura 19. Ejemplo del clúster de personajes.

Correlación interna-externa Esta última subsección responde a si las métricas intrínsecas guardan alguna relación clara con las extrínsecas. Sobre los resultados de E8a y E8b se calcula la correlación de Pearson entre la silueta y el ARI a nivel de celda (cada celda es una combinación de extractor, reducción y agrupador, agregada sobre sus repeticiones).

En E8a (estilos, 4 clases) la correlación es positiva y significativa: $r = +0.78$ ($n = 48$, $p < 10^{-7}$) sobre los agrupadores de k fijo (K-medias, aglomerativo y espectral con $k = 4$), y $r = +0.55$ ($n = 89$, $p < 10^{-7}$) al incluir las celdas HDBSCAN, de k libre. En E8b (autoría, 87 clases) el signo se invierte ($r = -0.54$ sobre las celdas con más de un clúster), pero este valor apenas tiene peso práctico, ya que todo el rango de ARI en E8b vive en una banda cercana al azar de modo que la correlación, sea cual sea su r o su p nominales, solo describe el orden de particiones que en el mayor de los casos es aleatorio. No indica que la silueta descarte una estructura de autoría recuperable, sino que tal estructura no existe a este nivel. La figura 20 representa ambas relaciones.

El factor común en ambas métricas es la granularidad de la verdad de referencia. La silueta recompensa pocos clústeres grandes y cohesivos. Con verdad de grano grueso (4 estilos) eso coincide con lo que el ARI reconoce como correcto, y el valor de la silueta suele ser defendible cuando k está fijado. Con verdad de grano fino (87 autores) ocurre lo contrario: las celdas de mayor ARI son agrupamientos con $k = 87$, que por fuerza obtienen una silueta casi nula (87 clústeres

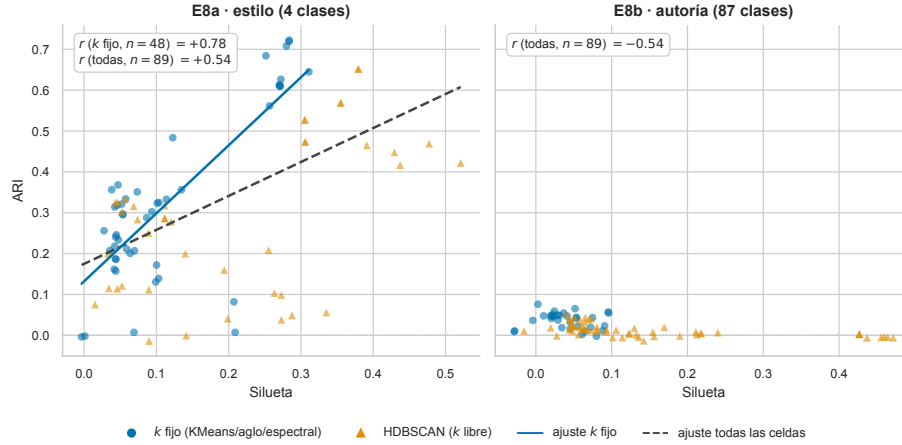


Figura 20. Silueta frente a ARI por ejecución en E8a (estilo) y E8b (autoría). Las rectas son ajustes por mínimos cuadrados sobre los agrupadores de k fijo (azul, continua) y sobre todas las celdas (gris, discontinua); en E8b se omiten porque el ARI no se despega del azar.

diminutos no pueden ser cohesivos), mientras que las de mayor silueta son las particiones degeneradas de HDBSCAN ($k = 2$ o $k = 3$) con un ARI nulo.

En resumen, la silueta es una métrica intrínseca adecuada para ordenar configuraciones a granularidad de estilo, pero no de autoría. La interpretación es la ya anticipada en calibración: las CNN entrenadas en ImageNet agrupan por apariencia genérica (textura, color, composición), y esa estructura es lo que la silueta detecta como conjuntos bien definidos, sin codificar la información estilística del dominio. Por eso su liderazgo intrínseco en E1 a $N = 6416$ no se traduce en ventaja supervisada en E8a: la silueta mide cohesión de apariencia, no identidad estilística.

La consecuencia práctica es la elección del extractor base: la cabeza estilística de DINOv2, uno de los peores modelos por métricas intrínsecas en el experimento inicial, se escoge por su liderazgo en E8a y no por el ranking intrínseco de E1. Esta decisión es importante, puesto que haber escogido el extractor con mejores métricas intrínsecas a $N = 6416$ habría llevado a una configuración de peor rendimiento en la tarea que motiva el propio trabajo.

6 Conclusiones

Este trabajo planteaba como objetivo general diseñar y evaluar un sistema modular para el agrupamiento de imágenes de grafiti, con un enfoque específico en la evaluación del coste computacional y la calidad del agrupamiento en función del tamaño del conjunto de datos. Los resultados obtenidos permiten extraer con-

clusiones concretas sobre la escalabilidad y calidad de diferentes combinaciones de componentes, así como identificar áreas clave para trabajo futuro.

6.1 Resultados principales

En cuanto al coste computacional, el barrido de escalabilidad (E7) confirma que, a esta escala, la ingesta de imágenes es el proceso inevitable más costoso, aun con un escalado lineal. La búsqueda por similitud en el almacenamiento con SQLite se convierte en un proceso aun más costoso, con un escalado cuadrático, lo que lo convertiría en el cuello de botella de la arquitectura de no ser por la existencia de alternativas como PostgreSQL con índices HNSW, que obtienen los mismos resultados de calidad a una fracción del coste.

La etapa de reducción de dimensionalidad con UMAP muestra un comportamiento complejo, puesto que presenta dos regímenes de ejecución distintos dependiendo del tamaño de la muestra y de la activación de `pynndescent`. El agrupamiento con HDBSCAN es despreciable en coste a esta escala, mientras que otros agrupadores como el aglomerativo u OPTICS empiezan a encontrarse con limitaciones de memoria o tiempo respectivamente.

En cuanto a la calidad del agrupamiento, los barridos iniciales (de E1 a E5) se centraron en identificar combinaciones de componentes que ofrecieran un buen rendimiento en métricas intrínsecas de calidad de clústeres. La cabeza de estilo de DINOv2 obtuvo una silueta inferior a las CNN preentrenadas en ImageNet, pero su desempeño en la evaluación supervisada de estilo (E8a) fue significativamente superior, lo que valida su elección a pesar de las mediocres métricas intrínsecas. La reducción de dimensionalidad probó ser un elemento indispensable para obtener agrupamientos de mejor calidad y para reducir el coste de ejecución posterior. Finalmente, el agrupamiento con HDBSCAN mostró un comportamiento robusto y consistente, con una configuración de `mcs=5` que ofrecía un buen compromiso entre calidad y número de clústeres obtenidos.

Por último, la evaluación supervisada (E8a–E8b) revela una fuerte asimetría entre los dos ejes de similitud evaluados. Sobre estilo (4 clases, 294 recortes), la cabeza específica de DINOv2 alcanza $\text{ARI} = 0.722$ y $\text{F1} = 0.806$, mientras que los mejores extractores sin entrenar no superan $\text{ARI} = 0.368$. Sobre autoría (87 autores, 185 recortes), todos los extractores caen en una banda $\text{ARI} = 0.041\text{--}0.076$, cercana al azar. La causa más probable de la falta de éxito en E8b es la combinación de un cambio de dominio Cuenca→Salamanca, un número insuficiente de recortes por autor y la escasa variabilidad intraautor en el subconjunto disponible. No es un resultado que invalide las cabezas de autoría, sino que indica que el conjunto de datos anotado es insuficiente para entrenar representaciones útiles para esta tarea.

6.2 Trabajo futuro

Aunque el trabajo ha permitido obtener las conclusiones antes destacadas, queda abierta la vía para la extensión del trabajo a partir de la superación de algunas limitaciones presentes. La más evidente es el estrecho rango de N evaluado

en el barrido de escalabilidad. Con solo 6416 imágenes reales, el barrido abarca ≈ 1.4 décadas en escala logarítmica, insuficiente para distinguir con seguridad $O(n \log n)$ de $O(n^2)$ en las etapas intermedias. Esto sería relativamente sencillo de solucionar remuestreando con reemplazo los vectores DINOv2 ya calculados para aumentar el rango de N lo que fuese necesario, pero afectaría a los resultados de calidad obtenidos, puesto que los valores remuestreados no tienen significado real. Esta solución sería exclusivamente de coste y se etiquetaría como proyección, no como medición.

Tanto esta limitación como a la siguiente, la escasez de datos anotados por autoría, también podrían ser abordadas mediante una gran extensión del conjunto de datos. Este aumento no solo extendería el rango de N para el barrido de escalabilidad, sino que también permitiría obtener un conjunto más apropiado para entrenar las cabezas de autoría. Además, un conjunto de datos más grande también permitiría distintos enfoques para el *fine-tuning* de ambas cabezas, como por ejemplo el pre-entrenamiento no supervisado sobre el dataset completo y el uso de la supervisión de estilo y autoría solo en la fase final, lo que reduciría la dependencia de un conjunto anotado pequeño.

Por último, el sistema actual opera en modo «*batch*»: la ingesta es completa antes de que el agrupamiento comience. En escenarios reales las bases de datos de grafiti crecen de forma continua, y tanto HDBSCAN como UMAP carecen de una implementación incremental estable. Extender el sistema para soportar actualizaciones incrementales del índice y del espacio reducido puede ser una mejora importante para aplicar algunos de los resultados obtenidos a un sistema real.

Referencias

1. Fogaça, J., Brandão, T., Ferreira, J.C.: Deep Learning-Based Graffiti Detection: A Study Using Images from the Streets of Lisbon. *Applied Sciences* **13**(4) (January 2023) 2249
2. Taylor, M.F., Cordin, R., Njiru, J.: A twenty-first century graffiti classification system: A typological tool for prioritizing graffiti removal. *Crime Prevention and Community Safety* **12**(3) (July 2010) 137–155
3. Tokuda, E.K., Cesar, R.M., Silva, C.T.: Quantifying the presence of graffiti in urban environments. In: 2019 Ieee International Conference on Big Data and Smart Computing (bigcomp), New York, IEEE (2019) 405–408 Num Pages: 4 Series Title: International Conference on Big Data and Smart Computing Web of Science ID: WOS:000469779800067.
4. García García, M., Hernández, G., González-Arrieta, A., Rodríguez González, S.: Graffiti Identification Using Color Analysis: An Approach Based on K-Means Clustering. In Marreiros, G., Grande, L., Llerena, J.P., Conceição, L., Ko, H., Plaza, M., Ricca, M., eds.: Distributed Computing and Artificial Intelligence, Special Sessions II, 21st International Conference, Cham, Springer Nature Switzerland (2025) 323–328
5. He, K., Zhang, X., Ren, S., Sun, J.: Deep Residual Learning for Image Recognition (December 2015) arXiv:1512.03385 [cs.CV].

6. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I.: Learning Transferable Visual Models From Natural Language Supervision (February 2021) arXiv:2103.00020 [cs.CV].
7. Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.Y., Li, S.W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jegou, H., Mairal, J., Labatut, P., Joulin, A., Bojanowski, P.: DINOv2: Learning Robust Visual Features without Supervision (February 2024) arXiv:2304.07193 [cs.CV].
8. McInnes, L., Healy, J., Melville, J.: UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction (September 2020) arXiv:1802.03426 [stat.ML].
9. Ester, M., Kriegel, H.P., Sander, J., Xu, X.: A density-based algorithm for discovering clusters in large spatial databases with noise. In: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining. KDD'96, Portland, Oregon, AAAI Press (August 1996) 226–231
10. Campello, R.J.G.B., Moulavi, D., Sander, J.: Density-Based Clustering Based on Hierarchical Density Estimates. In Pei, J., Tseng, V.S., Cao, L., Motoda, H., Xu, G., eds.: Advances in Knowledge Discovery and Data Mining, Berlin, Heidelberg, Springer (2013) 160–172
11. Arthur, D., Vassilvitskii, S.: k-means++: the advantages of careful seeding. In: Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms. SODA '07, USA, Society for Industrial and Applied Mathematics (January 2007) 1027–1035
12. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs (August 2018) arXiv:1603.09320 [cs.DS].
13. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs (February 2017) arXiv:1702.08734 [cs.CV].
14. Fiel, S., Sablatnig, R.: Writer Identification and Retrieval Using a Convolutional Neural Network. In Azzopardi, G., Petkov, N., eds.: Computer Analysis of Images and Patterns, Cham, Springer International Publishing (2015) 26–37
15. Schroff, F., Kalenichenko, D., Philbin, J.: FaceNet: A Unified Embedding for Face Recognition and Clustering (March 2015)
16. Min, E., Guo, X., Liu, Q., Zhang, G., Cui, J., Long, J.: A Survey of Clustering With Deep Learning: From the Perspective of Network Architecture. Ieee Access **6** (2018) 39501–39514 Num Pages: 14 Web of Science ID: WOS:000441128300001.
17. Caron, M., Bojanowski, P., Joulin, A., Douze, M.: Deep Clustering for Unsupervised Learning of Visual Features (March 2019) arXiv:1807.05520 [cs.CV].
18. Van Gansbeke, W., Vandenhende, S., Georgoulis, S., Proesmans, M., Van Gool, L.: SCAN: Learning to Classify Images Without Labels. In Vedaldi, A., Bischof, H., Brox, T., Frahm, J.M., eds.: Computer Vision – ECCV 2020, Cham, Springer International Publishing (2020) 268–285
19. Ankerst, M., Breunig, M.M., Kriegel, H.P., Sander, J.: OPTICS: ordering points to identify the clustering structure. SIGMOD Rec. **28**(2) (June 1999) 49–60
20. Frey, B.J., Dueck, D.: Clustering by Passing Messages Between Data Points. Science **315**(5814) (February 2007) 972–976
21. Dempster, A.P., Laird, N.M., Rubin, D.B.: Maximum Likelihood from Incomplete Data Via the *EM* Algorithm. Journal of the Royal Statistical Society Series B: Statistical Methodology **39**(1) (September 1977) 1–22

22. Ng, A.Y., Jordan, M.I., Weiss, Y.: On spectral clustering: analysis and an algorithm. In: Proceedings of the 15th International Conference on Neural Information Processing Systems: Natural and Synthetic. NIPS'01, Cambridge, MA, USA, MIT Press (January 2001) 849–856
23. Müllner, D.: Modern hierarchical, agglomerative clustering algorithms (September 2011) arXiv:1109.2378 [stat.ML].
24. Aggarwal, C.C., Hinneburg, A., Keim, D.A.: On the Surprising Behavior of Distance Metrics in High Dimensional Space. In Van den Bussche, J., Vianu, V., eds.: Database Theory — ICDT 2001, Berlin, Heidelberg, Springer (2001) 420–434
25. Jolliffe, I.: Principal Component Analysis. Springer Series in Statistics. Springer-Verlag, New York (2002)
26. Schölkopf, B., Smola, A., Müller, K.R.: Nonlinear Component Analysis as a Kernel Eigenvalue Problem. *Neural Computation* **10**(5) (July 1998) 1299–1319
27. Tenenbaum, J.B., Silva, V.d., Langford, J.C.: A Global Geometric Framework for Nonlinear Dimensionality Reduction. *Science* **290**(5500) (December 2000) 2319–2323
28. Rousseeuw, P.J.: Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics* **20** (November 1987) 53–65
29. Caliński, T., Harabasz, J.: A dendrite method for cluster analysis. *Communications in Statistics* **3**(1) (January 1974) 1–27 _eprint: <https://doi.org/10.1080/03610927408827101>.
30. Davies, D., Bouldin, D.: A Cluster Separation Measure. *Pattern Analysis and Machine Intelligence, IEEE Transactions on* **PAMI-1** (May 1979) 224–227
31. Simonyan, K., Zisserman, A.: Very Deep Convolutional Networks for Large-Scale Image Recognition (April 2015) arXiv:1409.1556 [cs.CV].
32. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the Inception Architecture for Computer Vision (December 2015) arXiv:1512.00567 [cs.CV].
33. Howard, A., Sandler, M., Chu, G., Chen, L.C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q.V., Adam, H.: Searching for MobileNetV3 (November 2019) arXiv:1905.02244 [cs.CV].
34. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (June 2021) arXiv:2010.11929 [cs.CV].
35. Ultralytics: Ultralytics YOLO11 (September 2024)
36. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É.: Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* **12**(85) (2011) 2825–2830
37. Huang, D., Wang, C.D., Wu, J.S., Lai, J.H., Kwok, C.K.: Ultra-Scalable Spectral Clustering and Ensemble Clustering. *Ieee Transactions on Knowledge and Data Engineering* **32**(6) (June 2020) 1212–1226 Num Pages: 15 Web of Science ID: WOS:000531422700013.
38. Pan, S.J., Yang, Q.: A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering* **22**(10) (October 2010) 1345–1359
39. Howard, J., Ruder, S.: Universal Language Model Fine-tuning for Text Classification (May 2018) arXiv:1801.06146 [cs.CL].

40. Sculley, D.: Web-scale k-means clustering. In: Proceedings of the 19th international conference on World wide web. WWW '10, New York, NY, USA, Association for Computing Machinery (April 2010) 1177–1178
41. Fränti, P., Sieranoja, S.: K-means properties on six clustering benchmark datasets. *Applied Intelligence* **48**(12) (December 2018) 4743–4759
42. Ahmed, M., Seraj, R., Islam, S.M.S.: The k-means Algorithm: A Comprehensive Survey and Performance Evaluation. *Electronics* **9**(8) (August 2020) 1295
43. Hermans, A., Beyer, L., Leibe, B.: In Defense of the Triplet Loss for Person Re-Identification. *ArXiv* (March 2017)
44. Wu, C.Y., Manmatha, R., Smola, A.J., Krähenbühl, P.: Sampling Matters in Deep Embedding Learning. In: 2017 IEEE International Conference on Computer Vision (ICCV). (October 2017) 2859–2867 ISSN: 2380-7504.
45. Khosla, P., Teterwak, P., Wang, C., Sarna, A., Tian, Y., Isola, P., Maschinot, A., Liu, C., Krishnan, D.: Supervised Contrastive Learning (March 2021) *arXiv:2004.11362 [cs.LG]*.